AWS VPC PEERING PROJECT

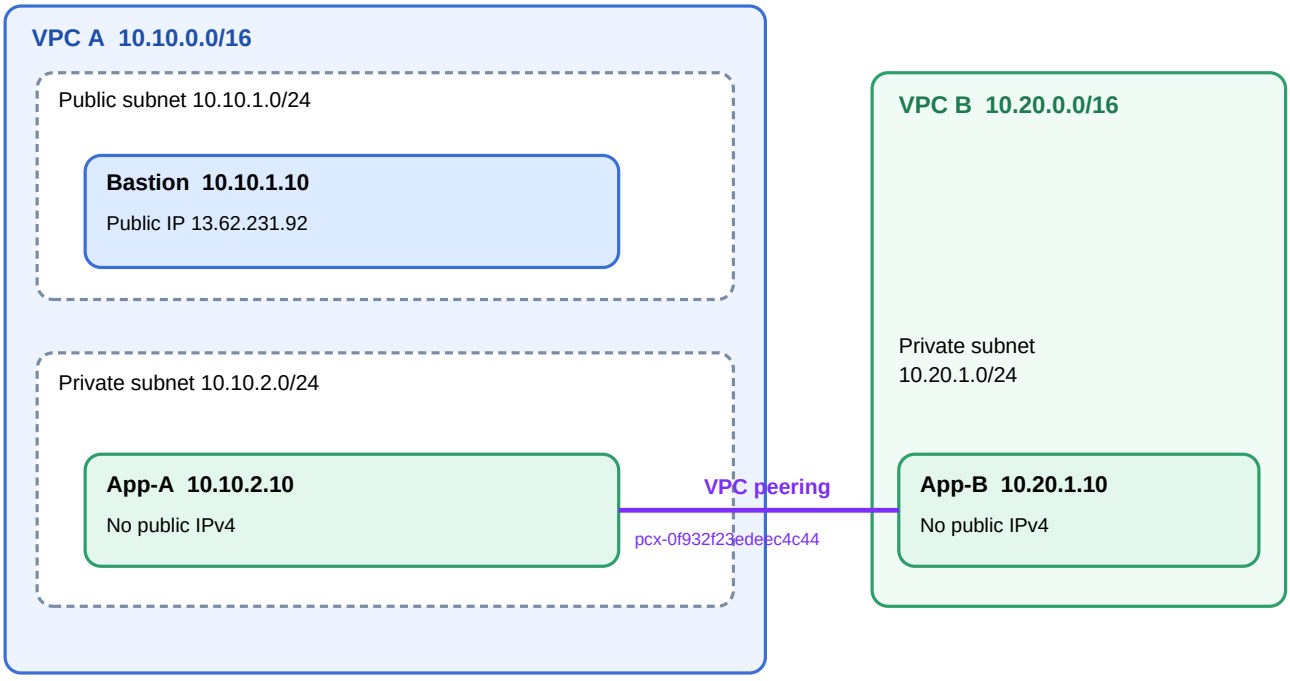
EWB Courses - Student Project Report

Prepared by: Himanshu Raj

Batch: AWS Internship Batch 1

Region: Europe (Stockholm) - eu-north-1

Date: 22 October 2026



Internet gateway attached to VPC A only. Private instances have no public IPv4.

Network design, connectivity tests, controlled failure and recovery evidence

1. Project Overview

This project connects two separate Amazon VPCs using VPC peering. A public bastion host in VPC A is used for SSH administration of two private instances (App-A in VPC A and App-B in VPC B). Route tables on both sides and security-group rules control the traffic. Connectivity was tested in both directions, App-B was reached over SSH through the bastion, and a controlled failure (removing the ICMP Echo Request rule) was followed by recovery.

Item	Configured value
AWS Region	eu-north-1 (Europe - Stockholm), Availability Zone eu-north-1a
VPC A	ewb-vpc-a - 10.10.0.0/16
VPC B	ewb-vpc-b - 10.20.0.0/16
Peering	ewb-a-b-peer - pcx-0f932f23edeec4c44 - Active (requester VPC A, accepter VPC B)
Bastion	ewb-bastion - private 10.10.1.10 - public 13.62.231.92
App-A	ewb-app-a - 10.10.2.10 - no public IPv4
App-B	ewb-app-b - 10.20.1.10 - no public IPv4
Tags	Name, Project = EWB-VPC-Peering, Student = HimanshuRaj

2. Resource Inventory (actual IDs)

Resource	Name	ID / address
VPC	ewb-vpc-a	vpc-0cb3f0a5f250ca09f (10.10.0.0/16)
VPC	ewb-vpc-b	vpc-00d22baa70fb398ce (10.20.0.0/16)
Subnet	ewb-a-public	subnet-05377fc487288647f (10.10.1.0/24)
Subnet	ewb-a-private	subnet-0f47a13fd27200d4a (10.10.2.0/24)
Subnet	ewb-b-private	subnet-04f549bcabaa370fc (10.20.1.0/24)
Internet gateway	ewb-igw-a	igw-01ffa935ed1819183 (attached to ewb-vpc-a)
Route table	ewb-rt-a-public	rtb-00d9cd33105b98e4e
Route table	ewb-rt-a-private	rtb-087dd3b90519586f6
Route table	ewb-rt-b-private	rtb-0a9e39de5b26af30a
Peering	ewb-a-b-peer	pcx-0f932f23edeec4c44
Security group	ewb-sg-bastion	sg-0fa9766674012cefc
Security group	ewb-sg-app-a	sg-01369347847aafe5b
Security group	ewb-sg-app-b	sg-060ea101e9e7a183d
Instance	ewb-bastion	i-0dff1a00547e05f5e - 10.10.1.10 + 13.62.231.92
Instance	ewb-app-a	i-097824f084a6c34db - 10.10.2.10
Instance	ewb-app-b	i-0b55fa48ed3ffe2ac - 10.20.1.10

3. Subnets and Route Tables

Route table	Subnet association	Routes
ewb-rt-a-public	ewb-a-public	10.10.0.0/16 local; 0.0.0.0/0 > Internet gateway; 10.20.0.0/16 > peering
ewb-rt-a-private	ewb-a-private	10.10.0.0/16 local; 10.20.0.0/16 > peering
ewb-rt-b-private	ewb-b-private	10.20.0.0/16 local; 10.10.0.0/16 > peering (return route)

The public route table also holds the peering route because the bastion opens the SSH connection to App-B (ProxyJump); the more specific 10.20.0.0/16 route is chosen over the internet default route.

4. Security Groups (inbound)

Security group	Rule	Source
ewb-sg-bastion	SSH TCP 22	My public IPv4 /32
ewb-sg-app-a	SSH TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-a	Custom ICMP - IPv4, Echo Request	10.20.1.10/32 (App-B)
ewb-sg-app-b	SSH TCP 22	10.10.1.10/32 (bastion private IP)
ewb-sg-app-b	Custom ICMP - IPv4, Echo Request	10.10.2.10/32 (App-A)

Outbound rules are the default allow-all; network ACLs are the default allow-all.

5. Test Results

Test	Where / action	Observed result
01 Baseline (no peering route)	App-A: ping -c 4 10.20.1.10 with the 10.20.0.0/16 peering route temporarily removed from ewb-rt-a-private	0 received, 100% packet loss. After adding the route back, the same ping gave 4 received, 0% loss
02 Forward ping	App-A: ping -c 4 10.20.1.10	4 received, 0% packet loss
03 Reverse ping	App-B: ping -c 4 10.10.2.10	4 received, 0% packet loss
04 Private SSH	Laptop: ssh ewb-app-b (ProxyJump via bastion)	Shell on ip-10-20-1-10; 'Last login from 10.10.1.10' shows the bastion hop
05 Isolation	EC2 instance details of App-A and App-B	Public IPv4 address is empty on both
06 Controlled failure	Removed the Echo Request rule (10.10.2.10/32) from ewb-sg-app-b; ping from App-A	0 received, 100% packet loss
07 Recovery	Added the same rule back (Custom ICMP - IPv4, Echo Request, 10.10.2.10/32); ping from App-A	4 received, 0% packet loss

Baseline note: the baseline was recorded after the peering connection was created, by temporarily removing the 10.20.0.0/16 peering route from ewb-rt-a-private. This gives the same no-route condition as before peering. The route was restored right after the test.

6. Controlled Failure Explanation

With the ICMP rule removed, the peering connection and all route tables were still correct, but the security group of App-B dropped the Echo Request, so the ping failed. SSH still worked because its rule was left in place. Restoring the exact same rule brought the ping back. This shows that an Active peering connection alone does not guarantee reachability: routes and security groups both have to allow the traffic.

7. Forward and Return Path

Forward (ping): App-A (10.10.2.10) > ewb-rt-a-private (10.20.0.0/16 to peering) > peering connection > App-B (10.20.1.10). **Return:** App-B replies using ewb-rt-b-private (10.10.0.0/16 to peering) back to App-A. Without the return route the reply would be lost. **Administration:** laptop > internet gateway > bastion (public subnet) > ProxyJump > App-A or App-B. The private key stays on the laptop and is never copied to an instance. VPC peering is one-to-one and non-transitive, and VPC B cannot use the internet gateway of VPC A through the peering connection.

8. What I Learned

- Planning non-overlapping VPC CIDRs and subnet layouts.
- Peering needs a request, an acceptance and routes on both sides.
- Security groups must allow only the traffic that is required (SSH from the bastion, ICMP between the apps).
- A private instance can be administered through a bastion with ProxyJump without giving it a public IP.
- Testing from the right host matters: the bastion is not allowed ICMP, so pings must be run from App-A or App-B.
- Removing and restoring a single rule is a clean way to prove which control blocks the traffic.

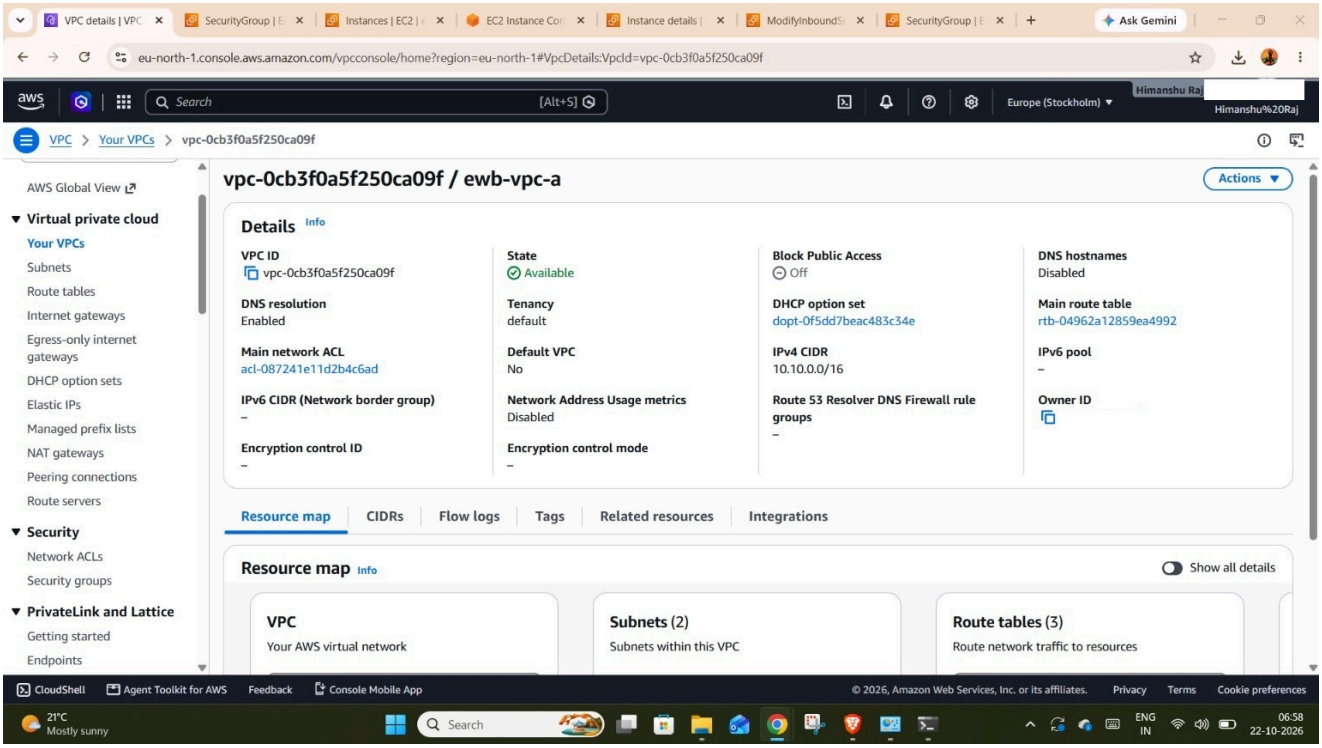
9. Cleanup Status

Cleanup was completed after the project evidence was captured. The cleanup screenshots on pages 15·22 document termination of the EC2 instances and deletion of the project security groups, subnets, route tables, VPC peering connection and internet gateway.

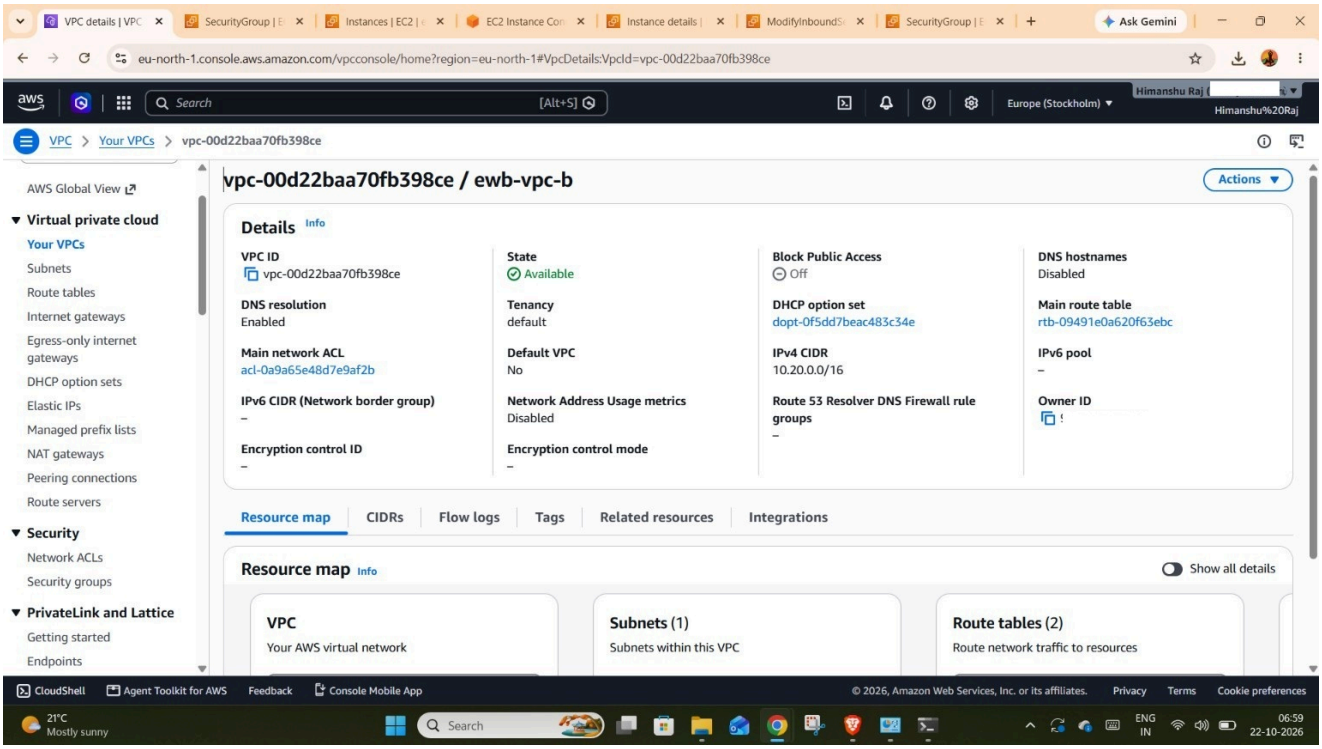
10. Evidence Screenshots

Screenshots are labeled with host, test and result. Private keys, passwords and access tokens are not included.

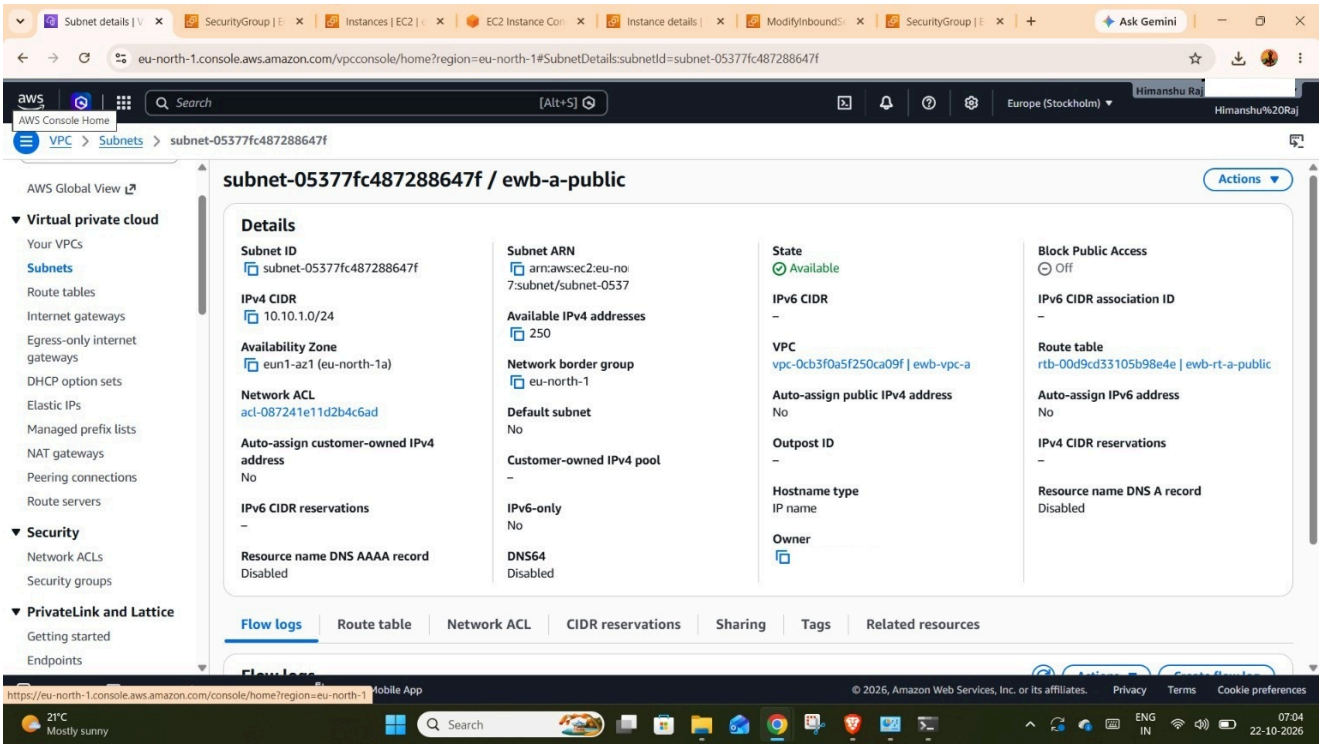
1. VPC A details (ewb-vpc-a, 10.10.0.0/16)



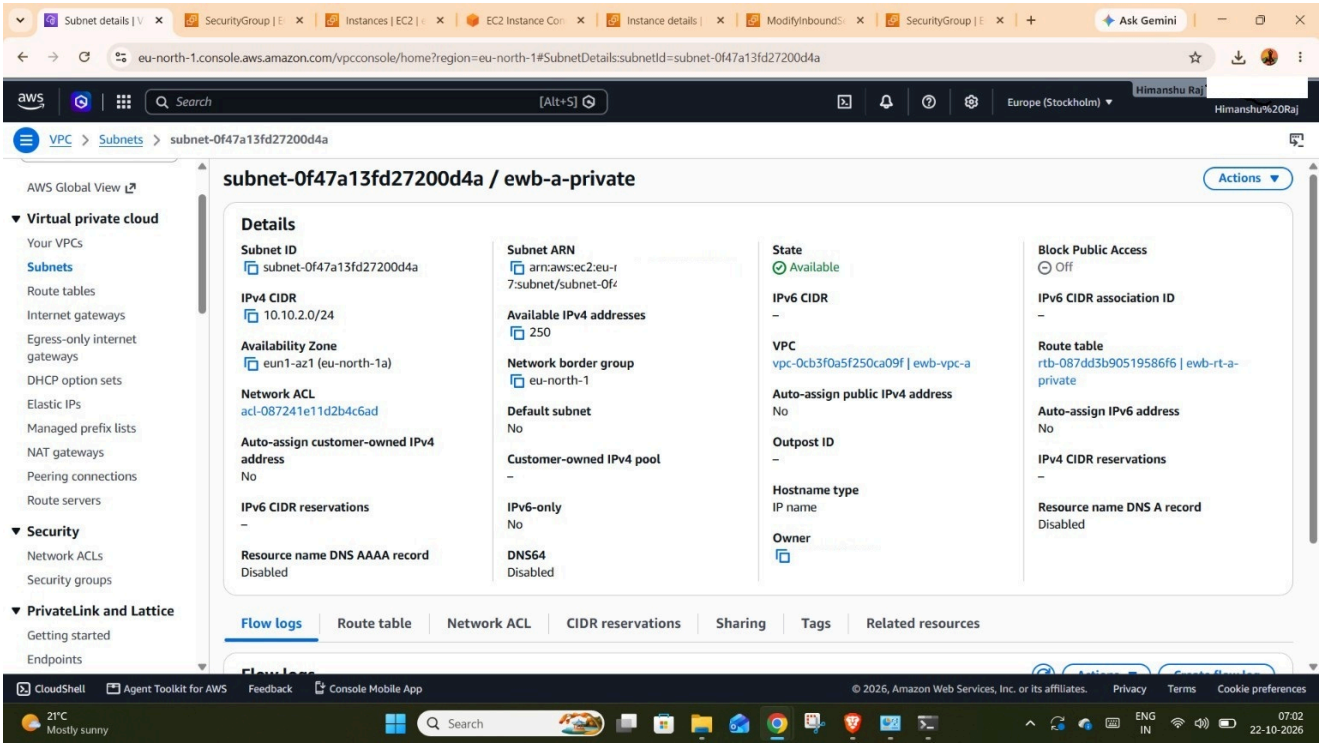
2. VPC B details (ewb-vpc-b, 10.20.0.0/16)



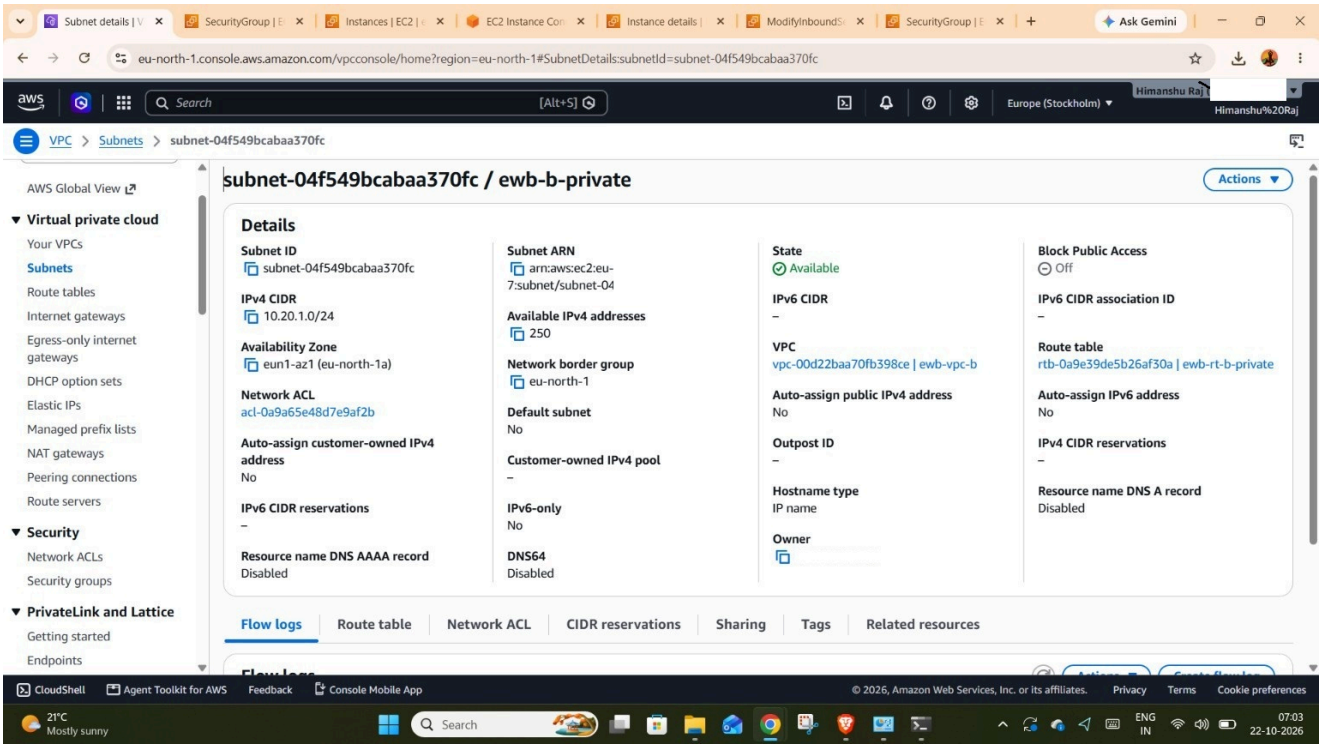
3. Subnet ewb-a-public (10.10.1.0/24)



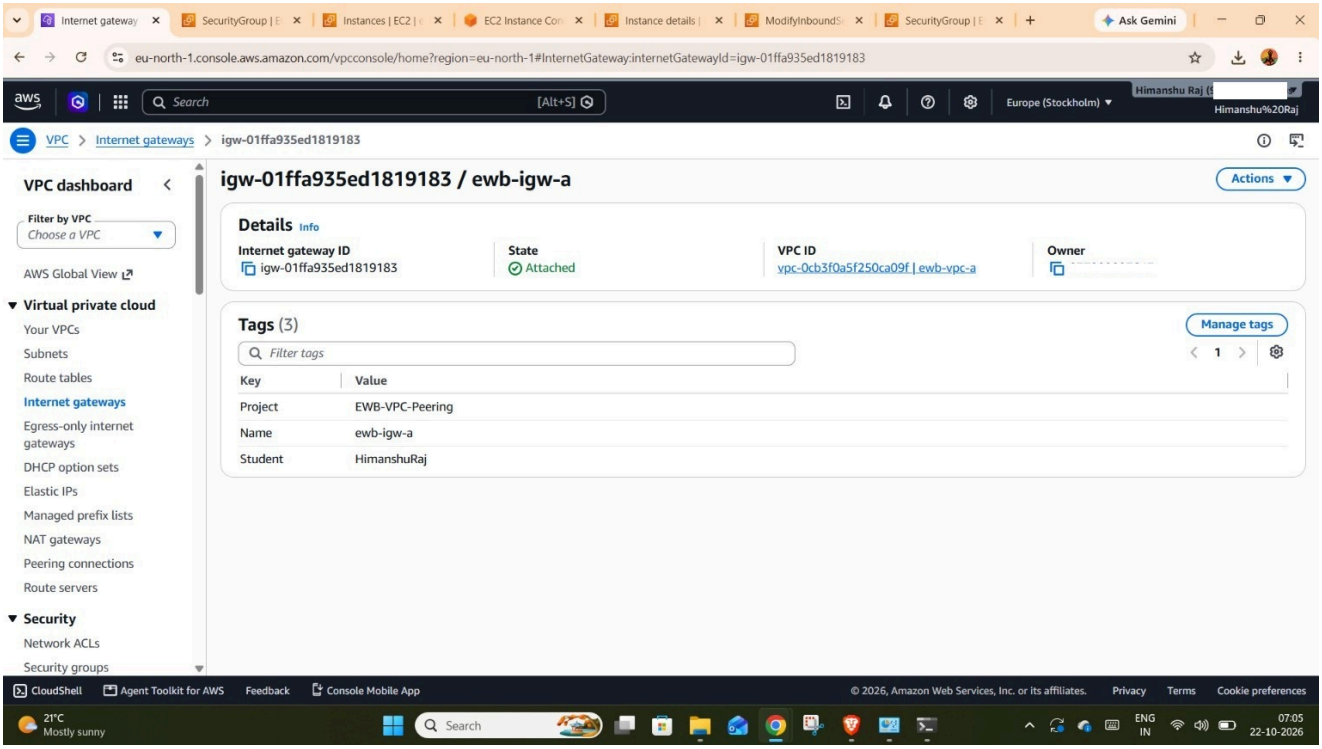
4. Subnet ewb-a-private (10.10.2.0/24)



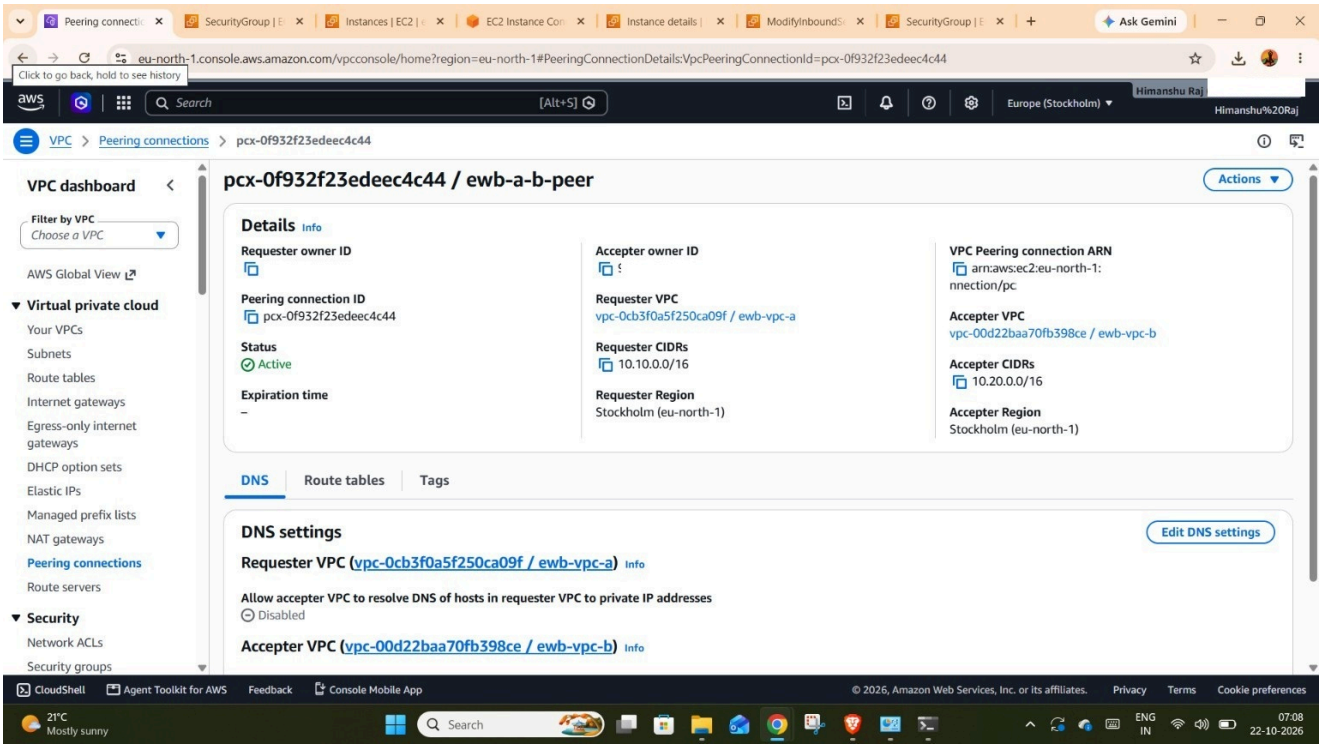
5. Subnet ewb-b-private (10.20.1.0/24)



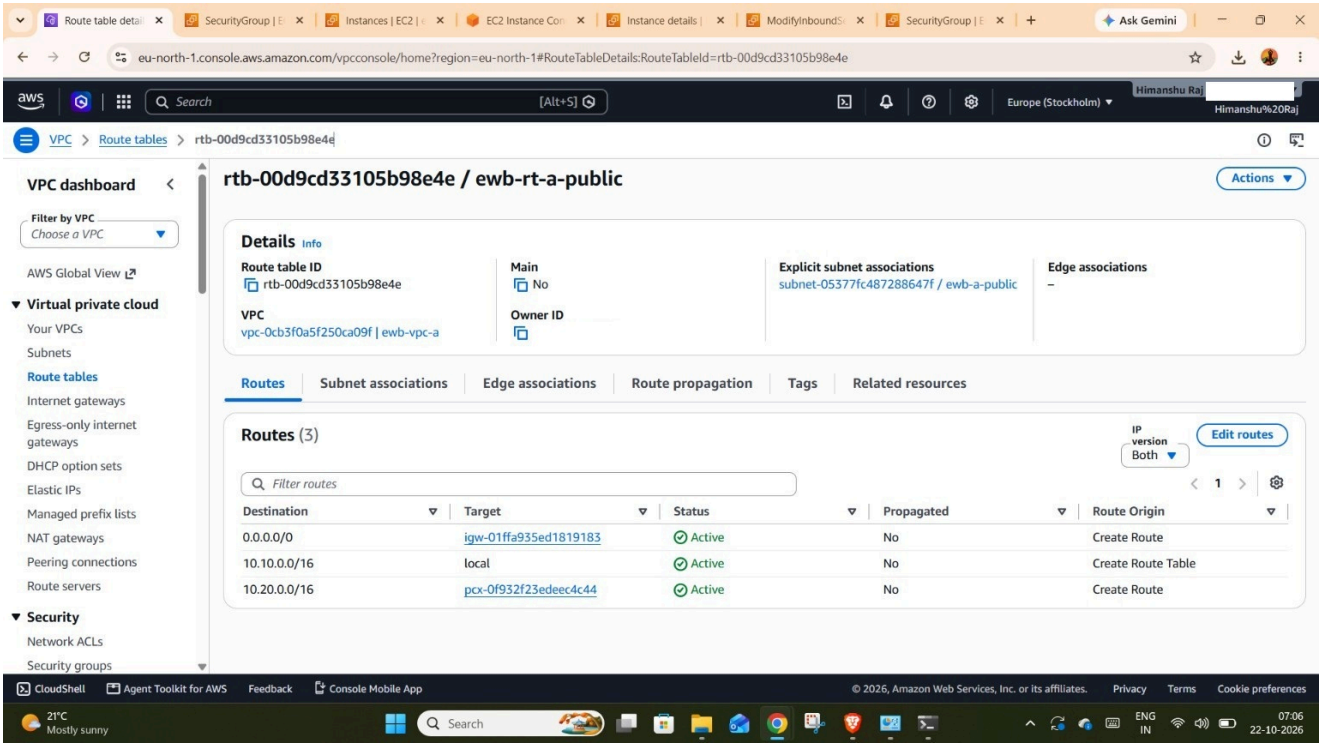
6. Internet gateway ewb-igw-a attached to ewb-vpc-a



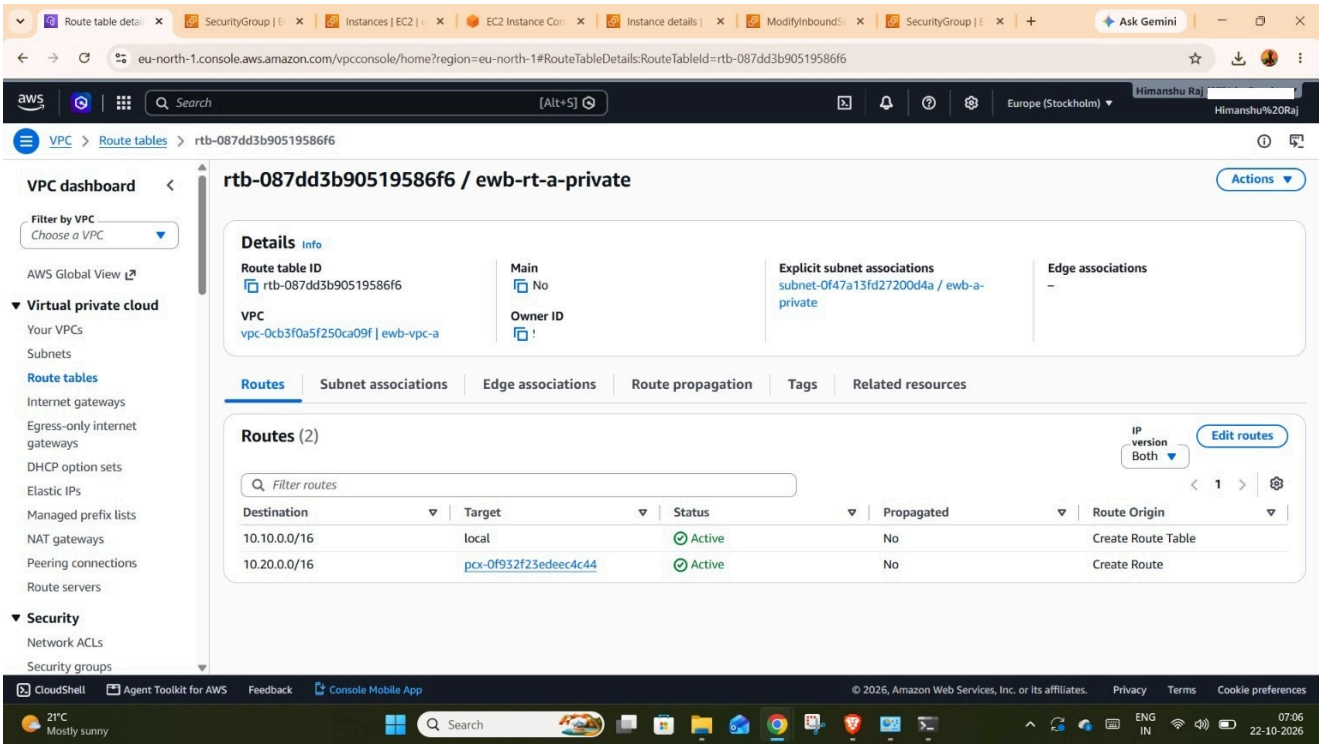
7. Peering connection ewb-a-b-peer - Active



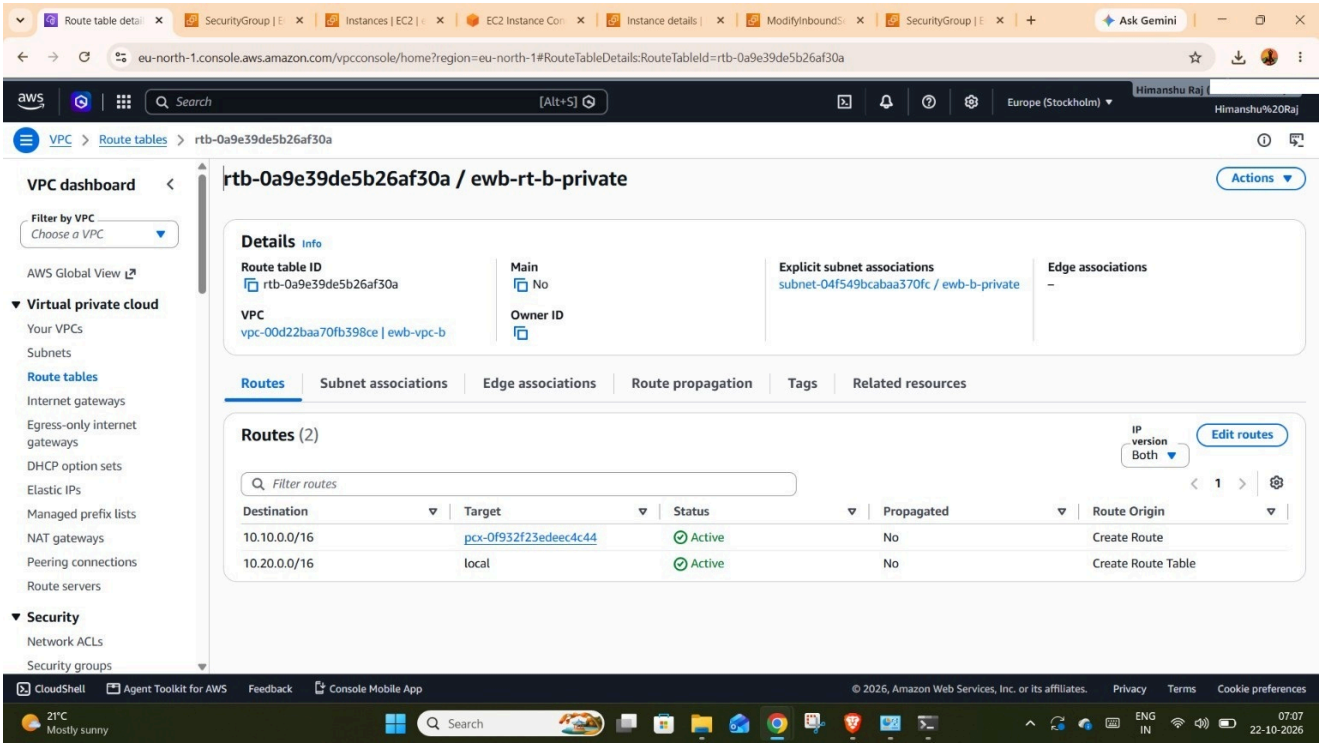
8. Route table ewb-rt-a-public - routes and association



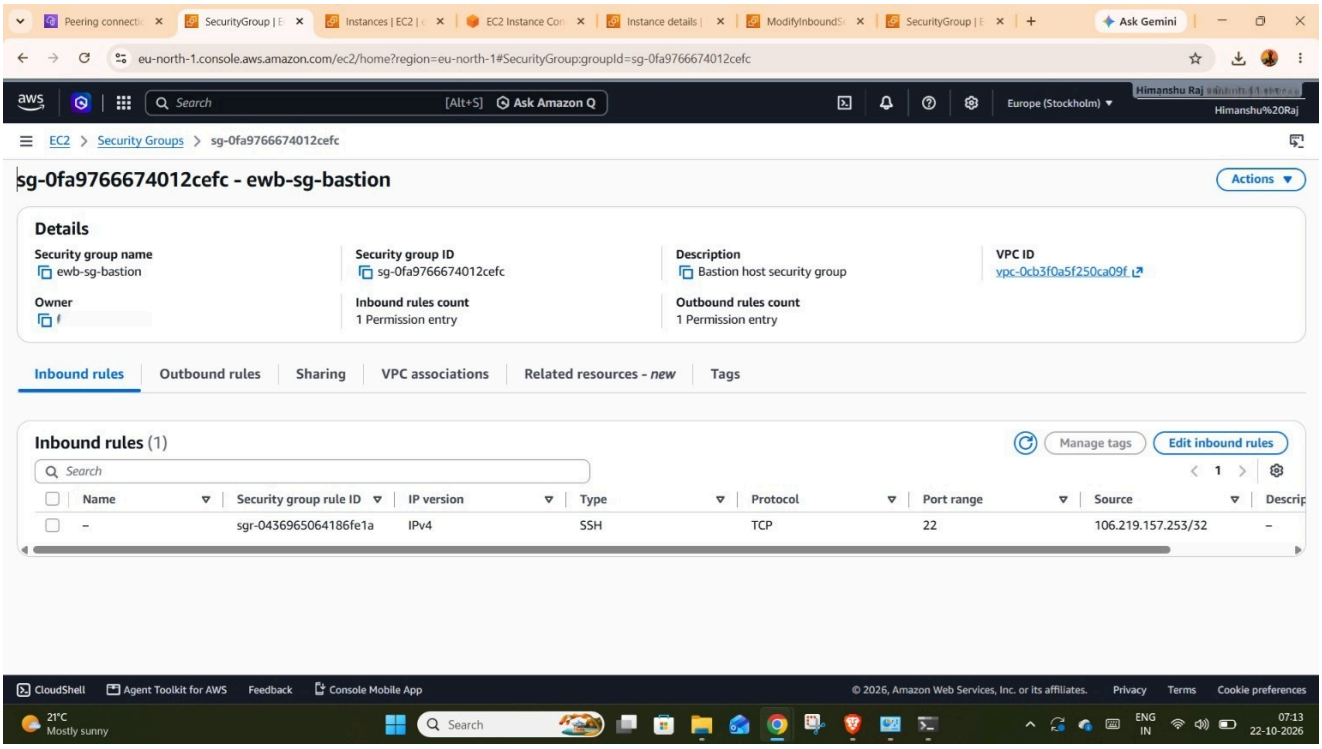
9. Route table ewb-rt-a-private - peering route



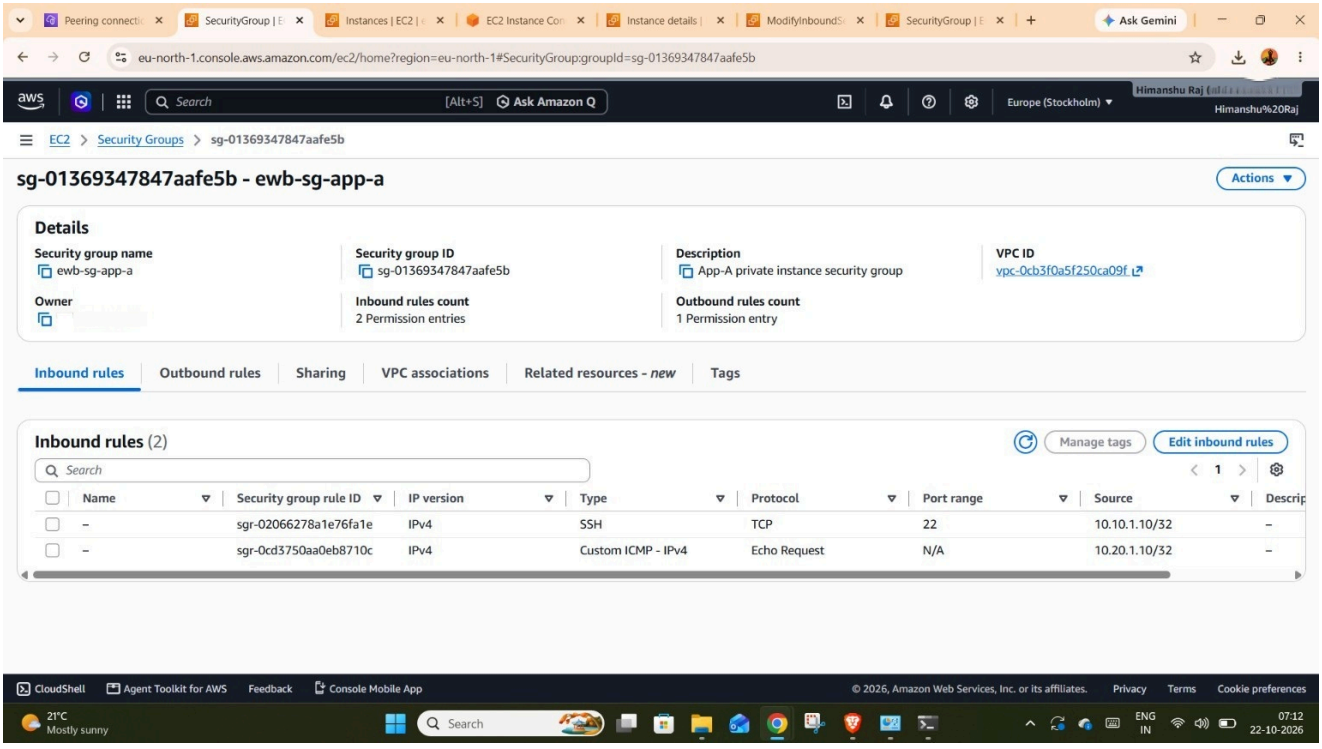
10. Route table ewb-rt-b-private - return route



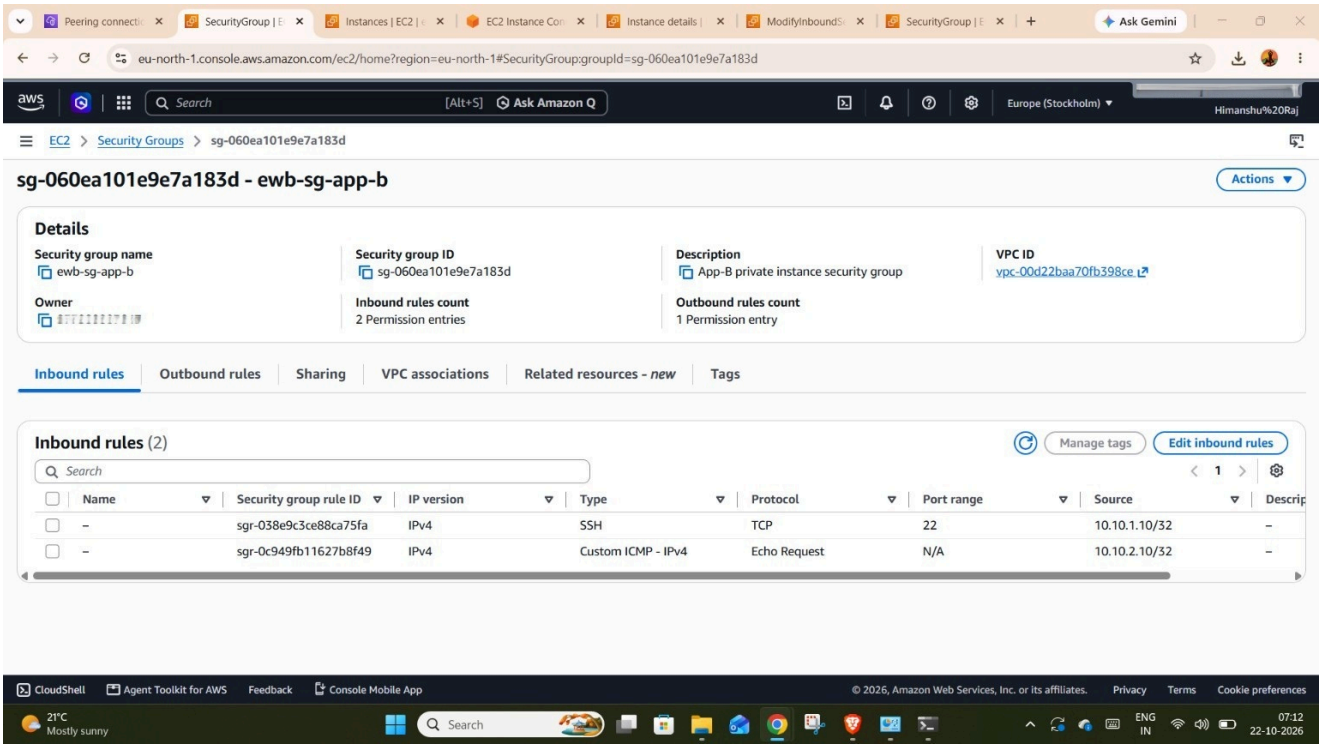
11. Security group ewb-sg-bastion - inbound rule



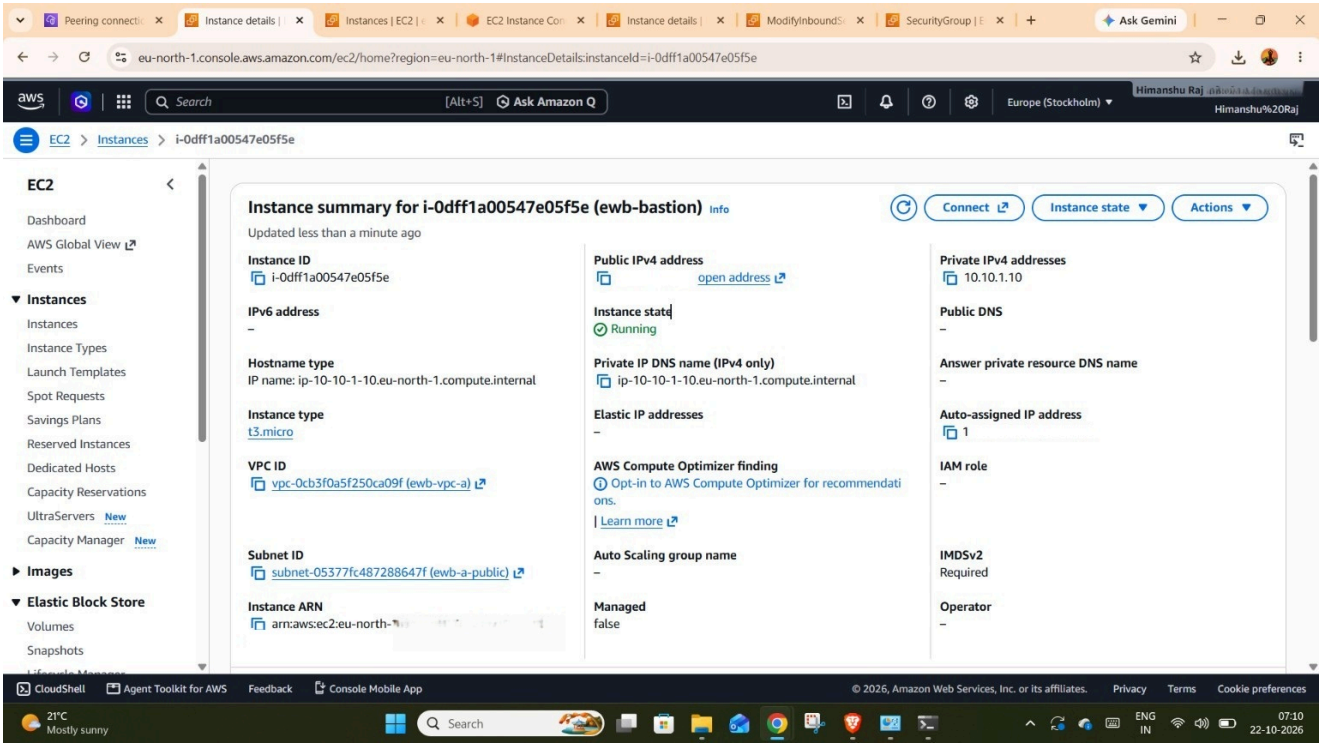
12. Security group ewb-sg-app-a - SSH and ICMP Echo Request



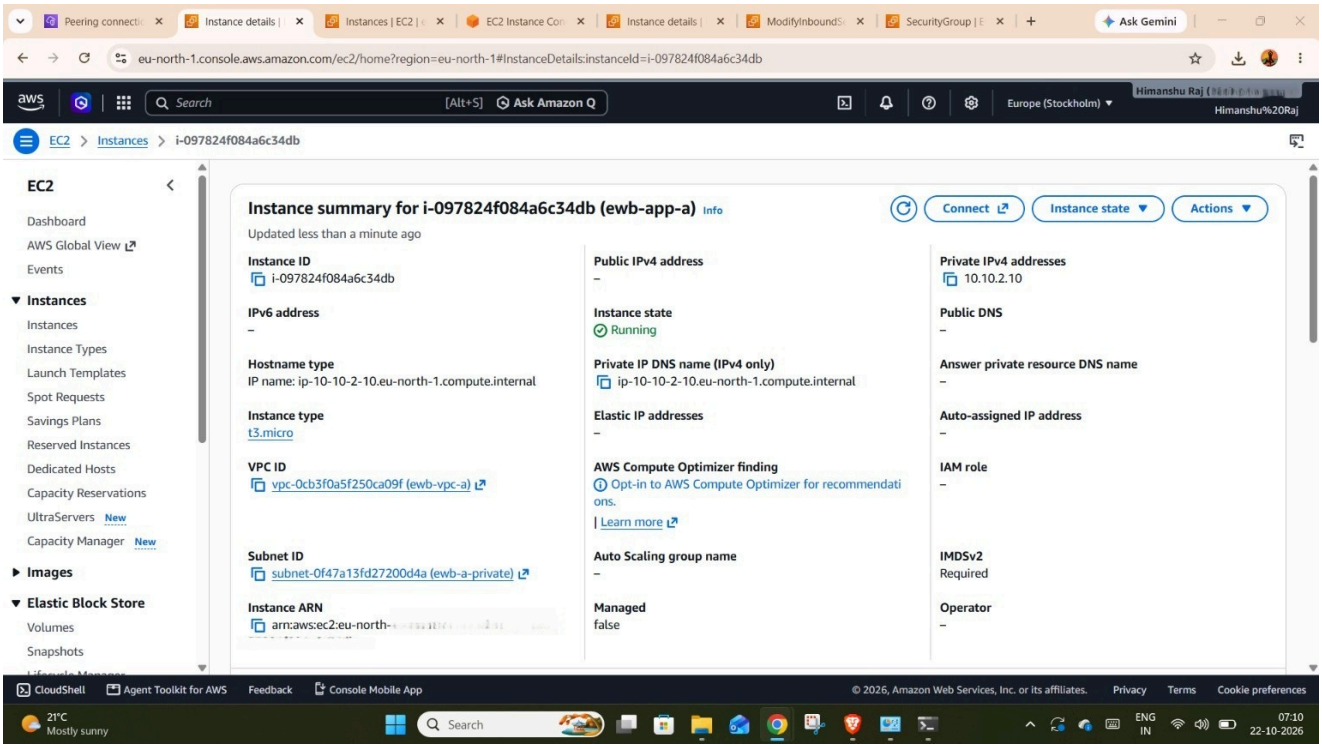
13. Security group ewb-sg-app-b - SSH and ICMP Echo Request



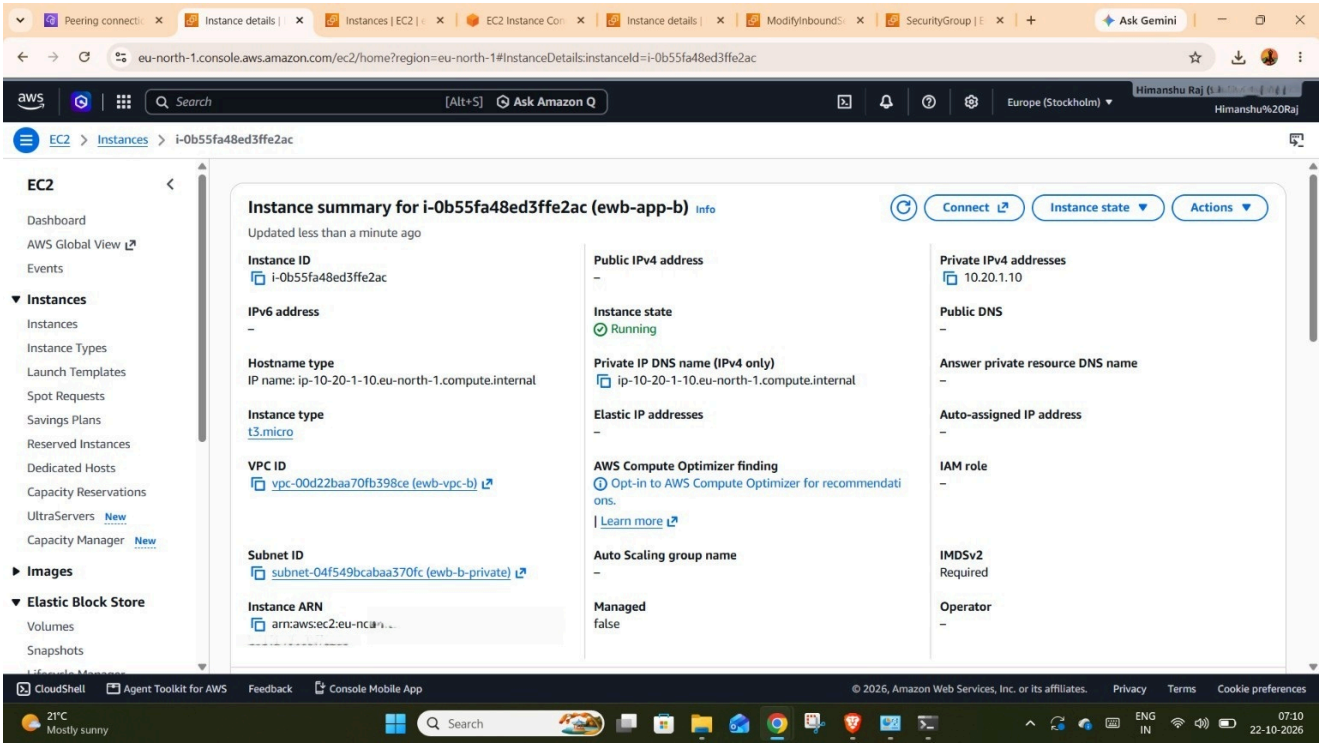
14. Instance ewb-bastion - public IPv4 13.62.231.92, private 10.10.1.10



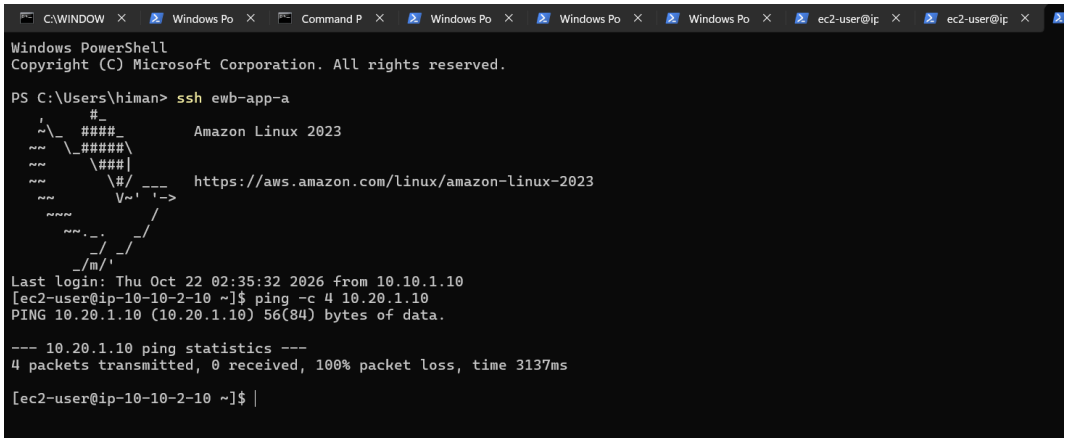
15. Test 05 Isolation - ewb-app-a has no public IPv4



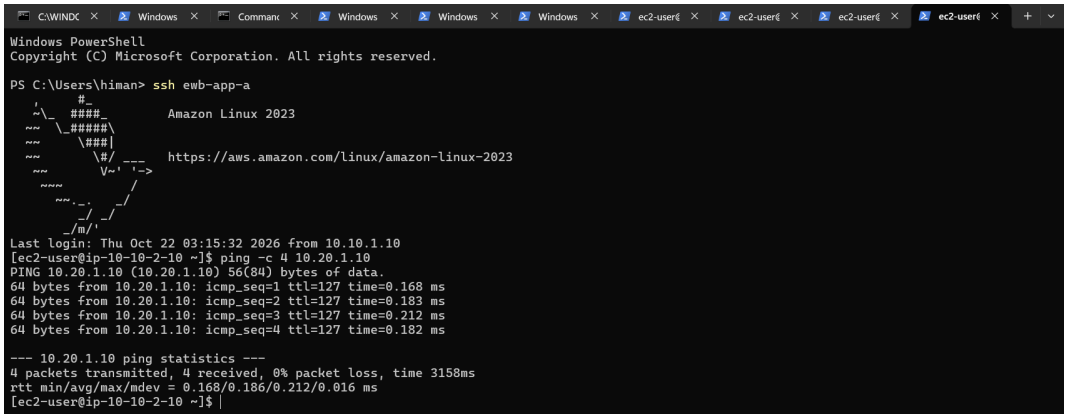
16. Test 05 Isolation - ewb-app-b has no public IPv4



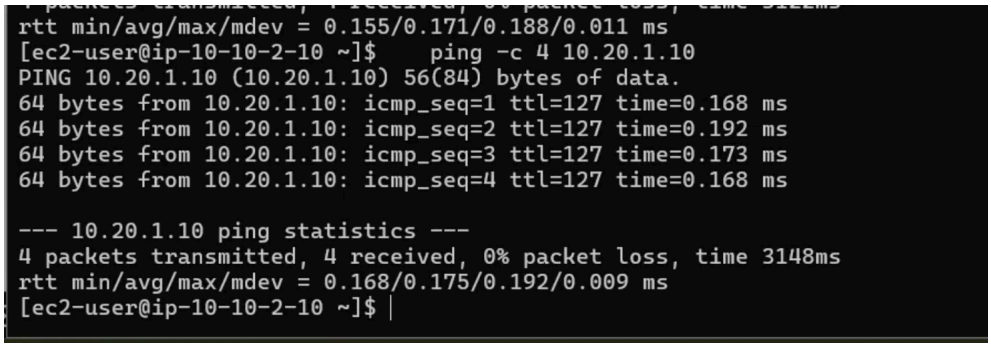
17. Test 01 Baseline - App-A (ip-10-10-2-10), peering route removed from ewb-rt-a-private: 0 received, 100% loss



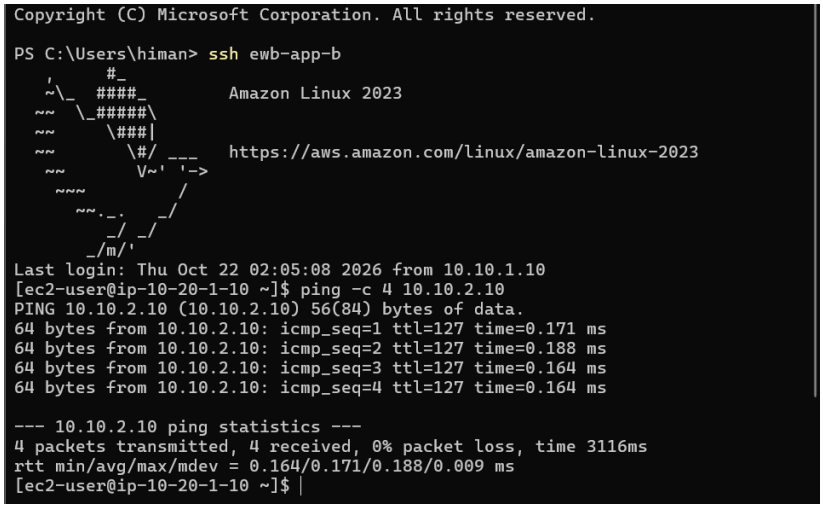
18. Test 01 Baseline restored - route added back, App-A ping: 4 received, 0% loss



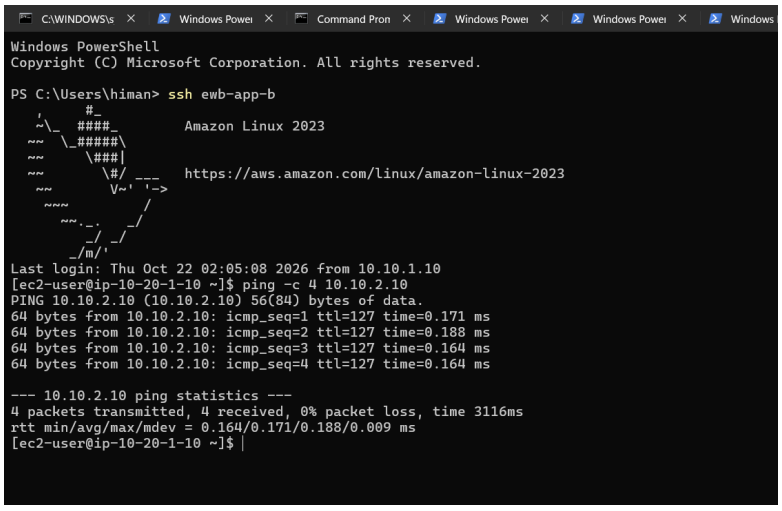
19. Test 02 Forward ping - App-A (ip-10-10-2-10) to 10.20.1.10: 4 received, 0% loss



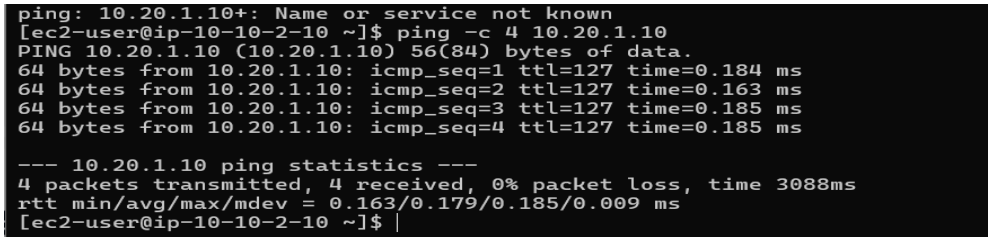
20. Test 03 Reverse ping - App-B (ip-10-20-1-10) to 10.10.2.10: 4 received, 0% loss



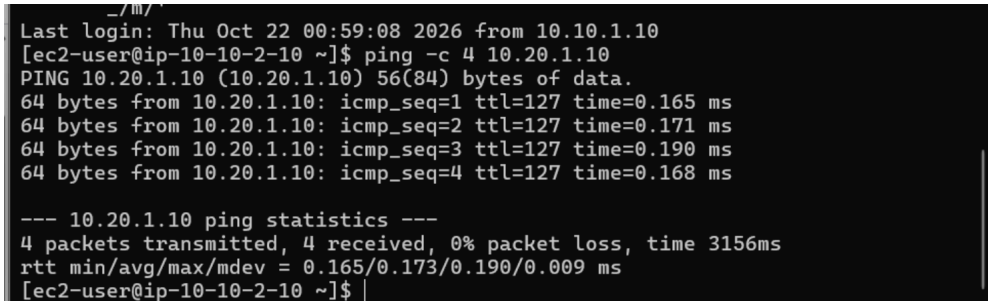
21. Test 04 Private SSH - laptop 'ssh ewb-app-b' through the bastion, then ping to App-A



22. Test 06 Controlled failure - ICMP rule removed from ewb-sg-app-b, App-A ping: 100% loss



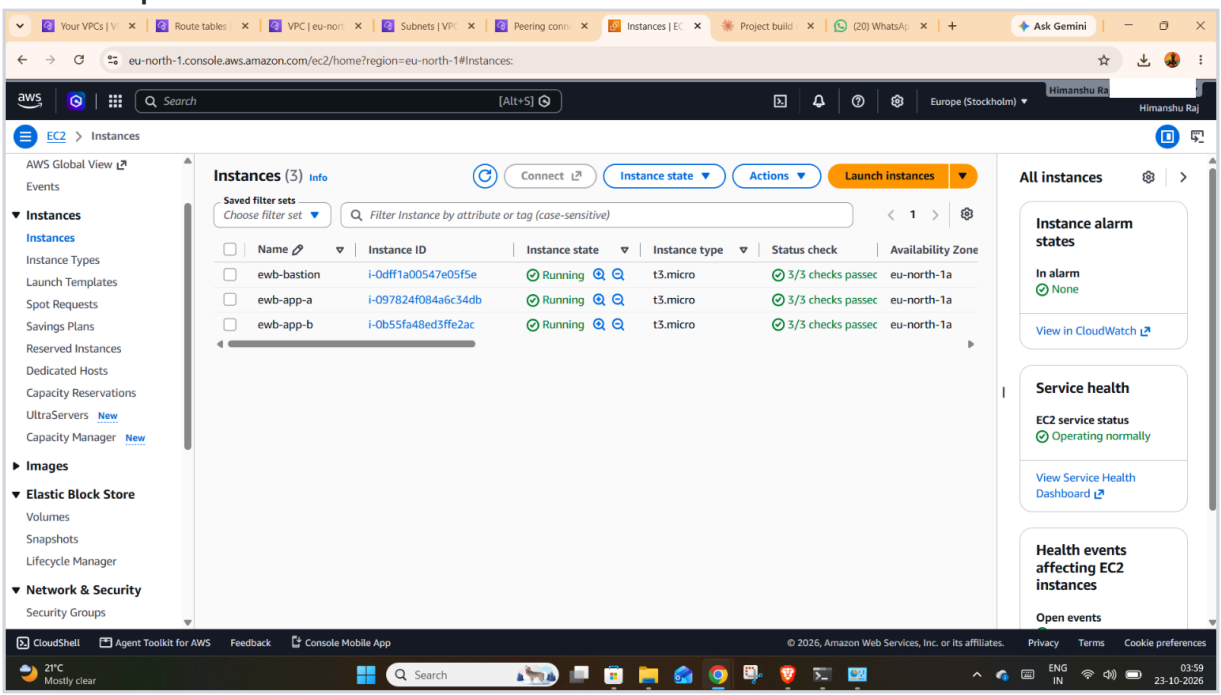
23. Test 07 Recovery - same rule restored, App-A ping: 4 received, 0% loss



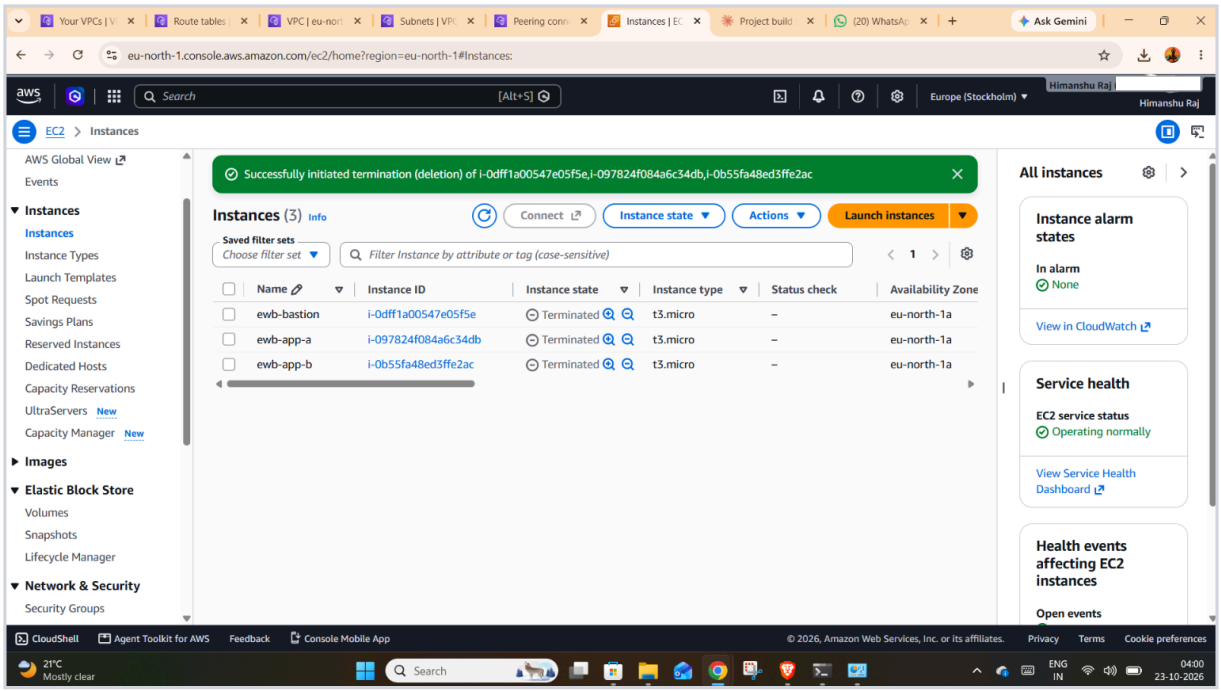
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

01. Before cleanup: EC2 instances listed



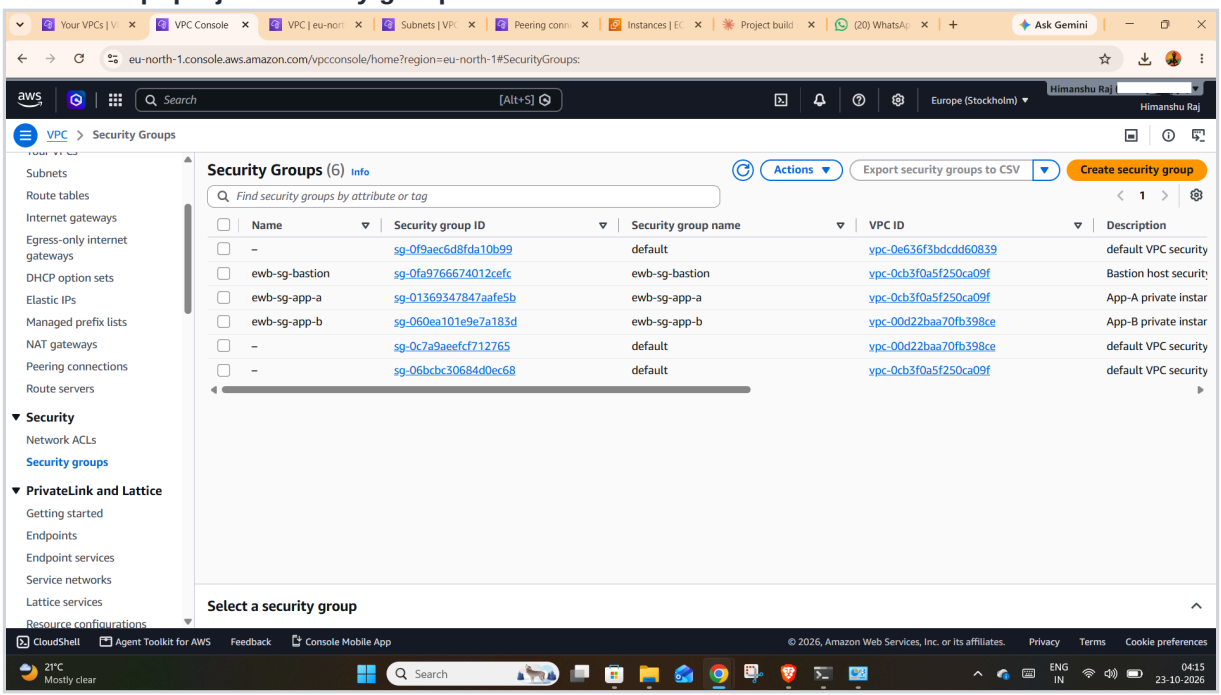
02. EC2 termination confirmation / instance state



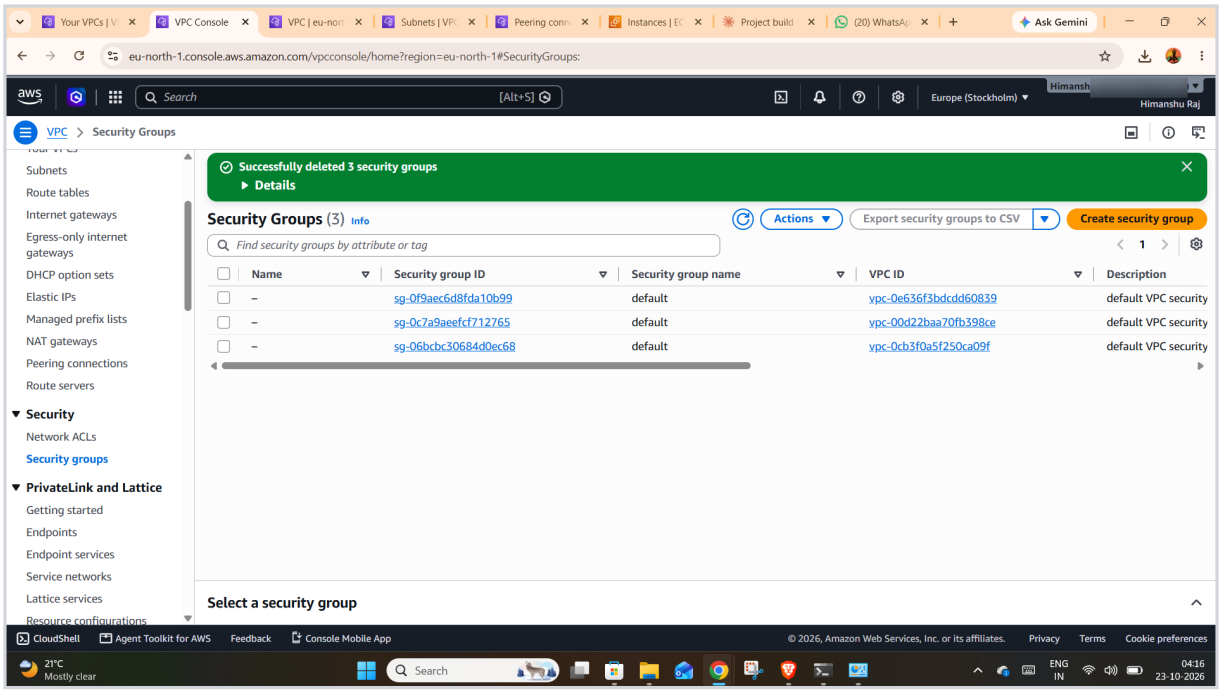
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

03. Before cleanup: project security groups listed



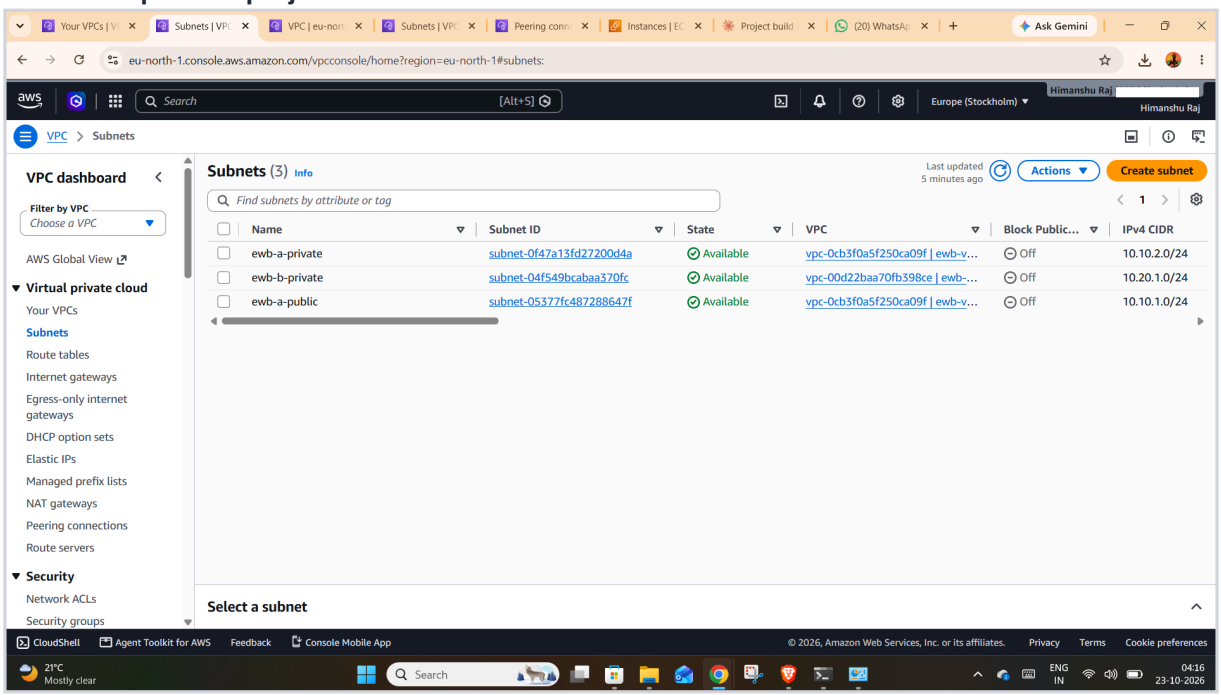
04. Project security groups deletion confirmation



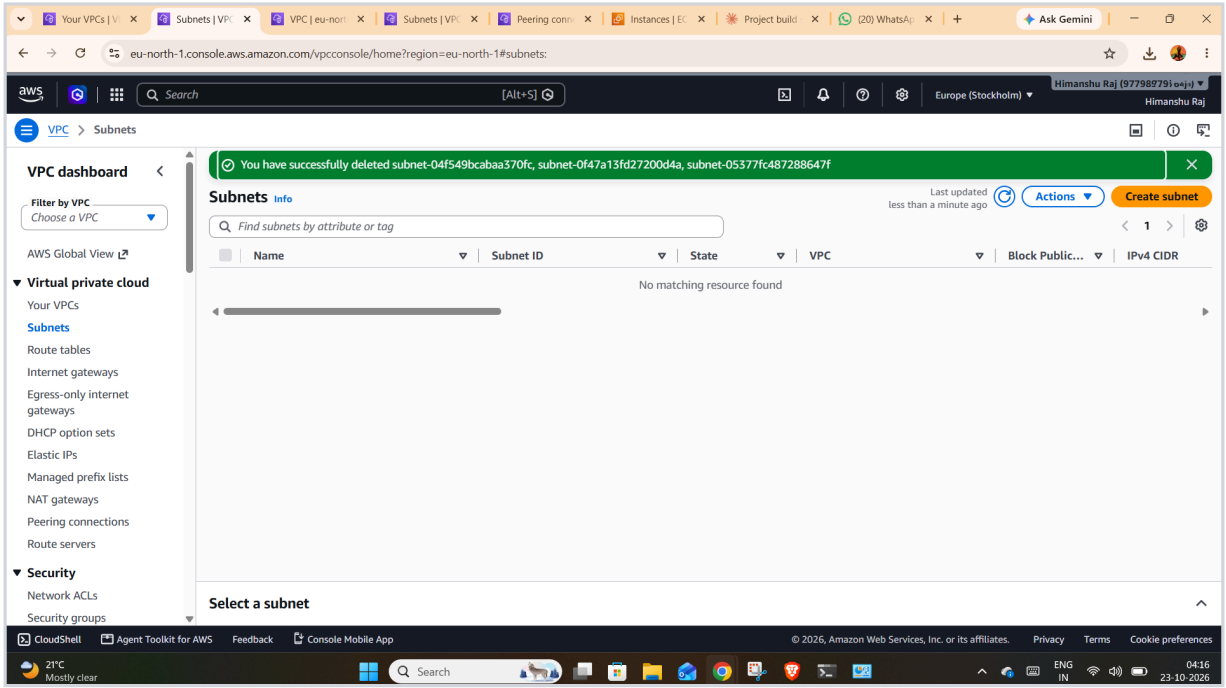
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

05. Before cleanup: three project subnets listed



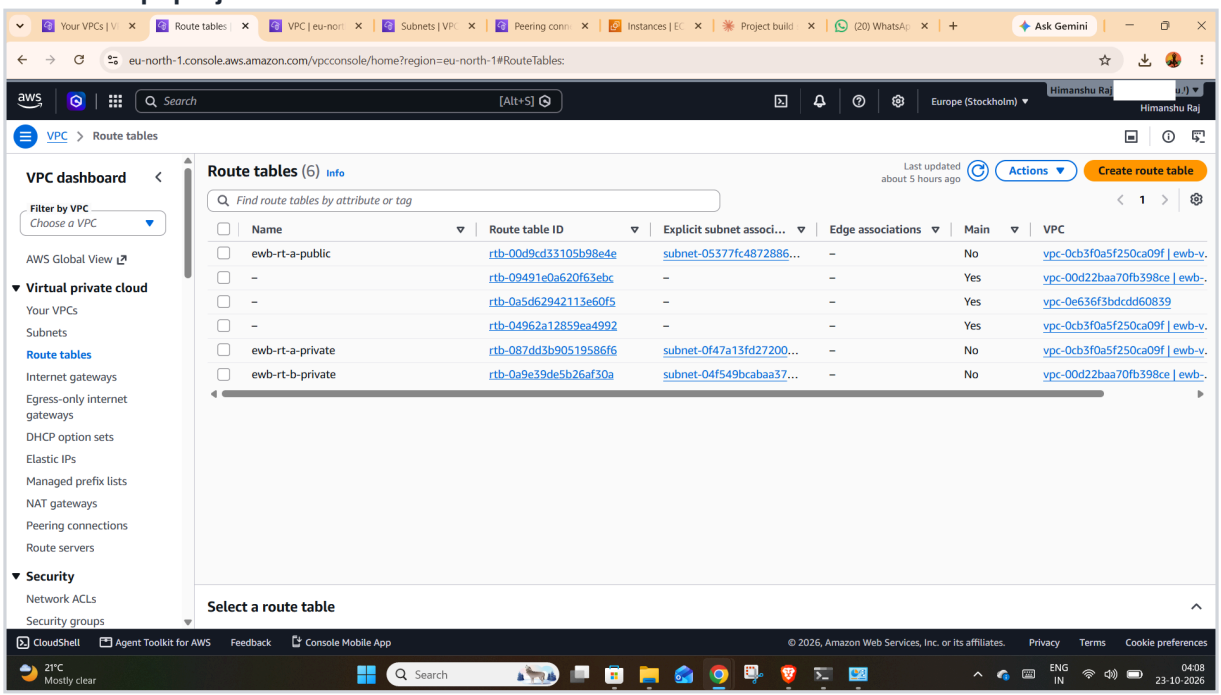
06. Deletion confirmation: project subnets



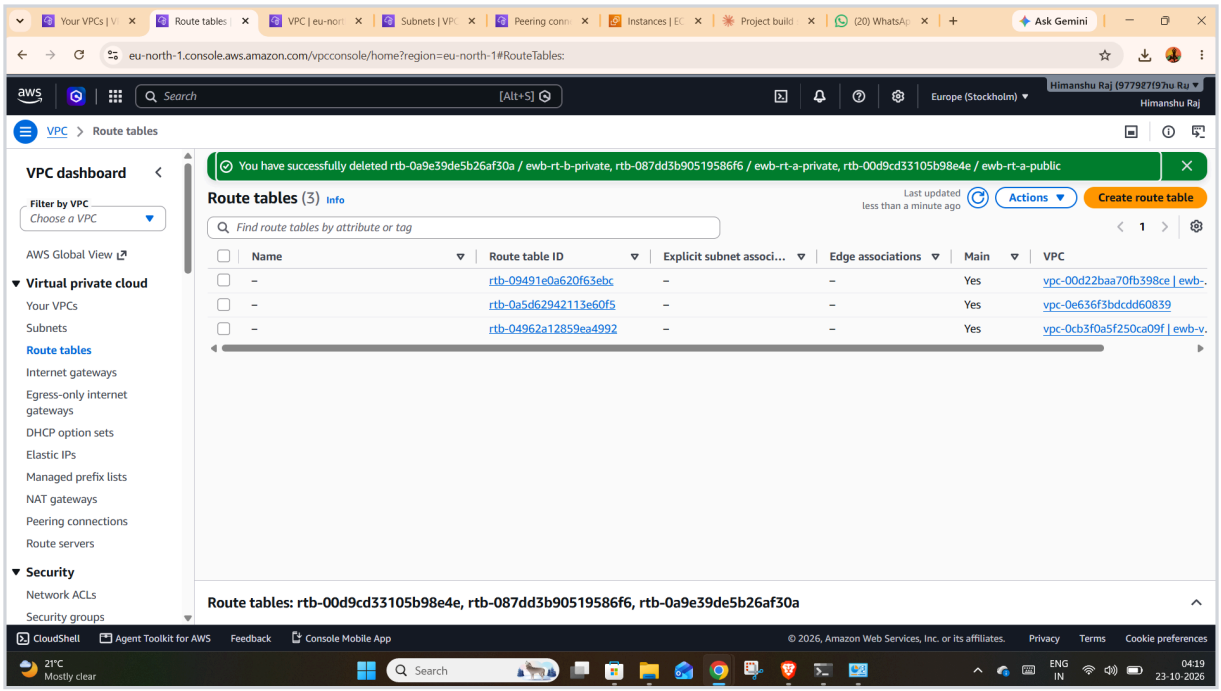
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

07. Before cleanup: project route tables listed



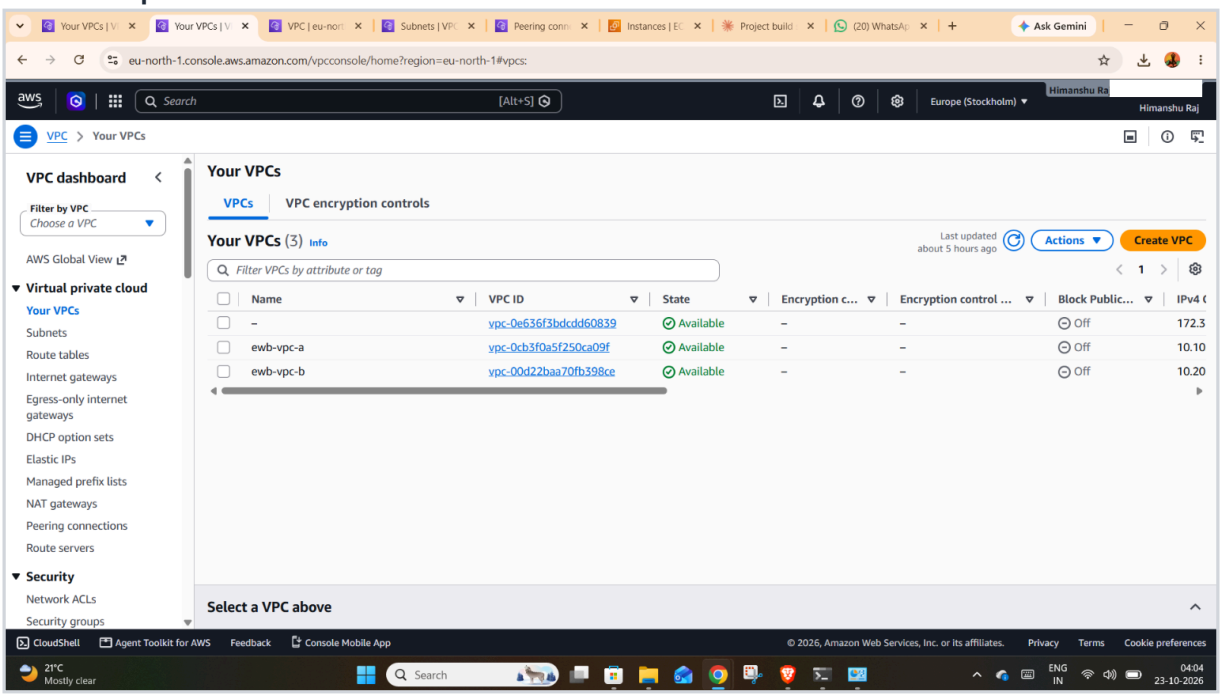
08. Deletion confirmation: project route tables



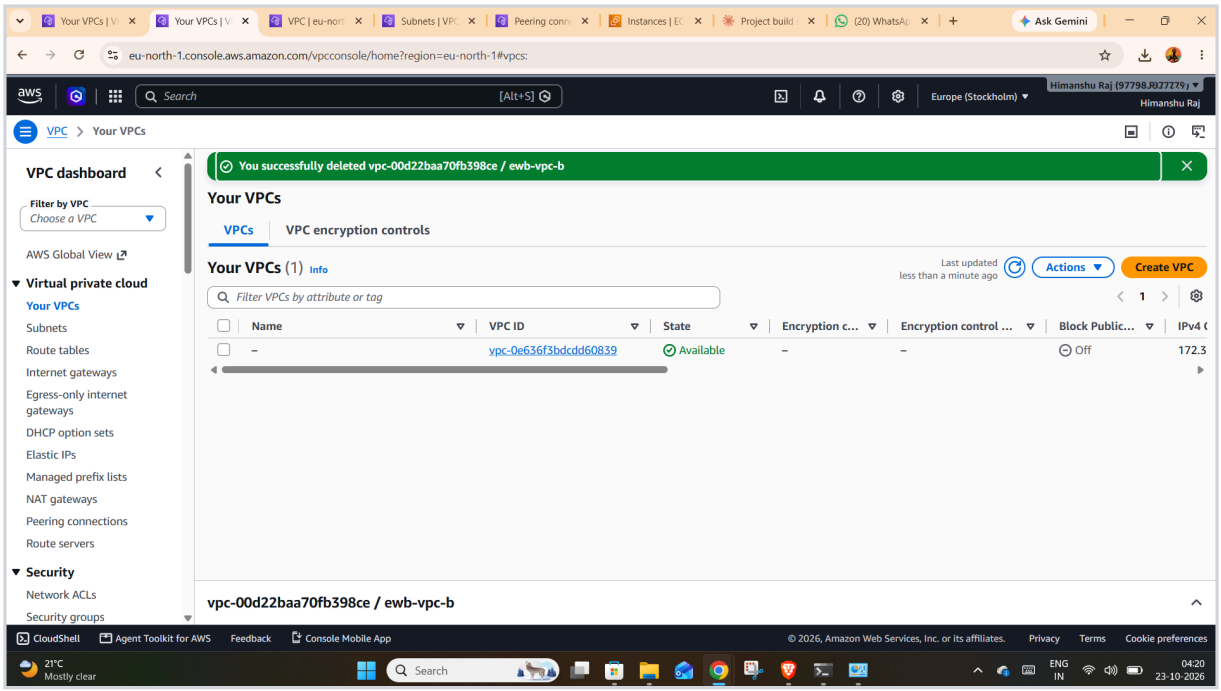
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

09. Before cleanup: VPC A and VPC B listed



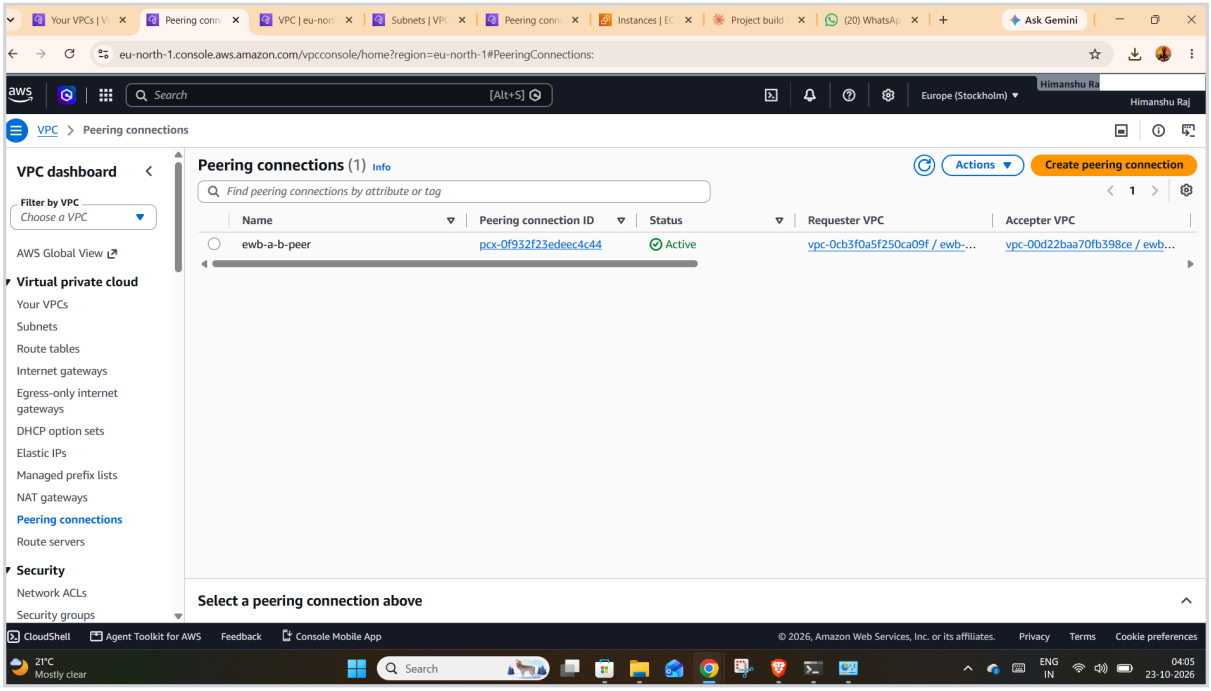
10. VPC B deletion confirmation



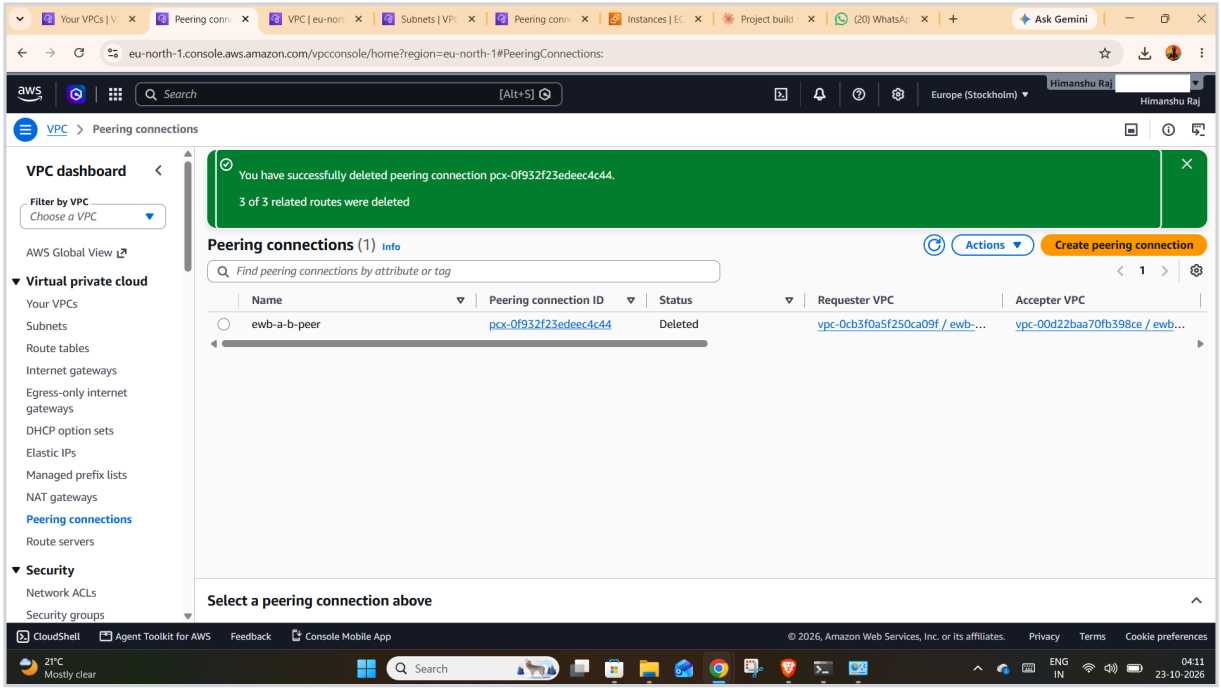
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

11. Before cleanup: VPC peering connection listed



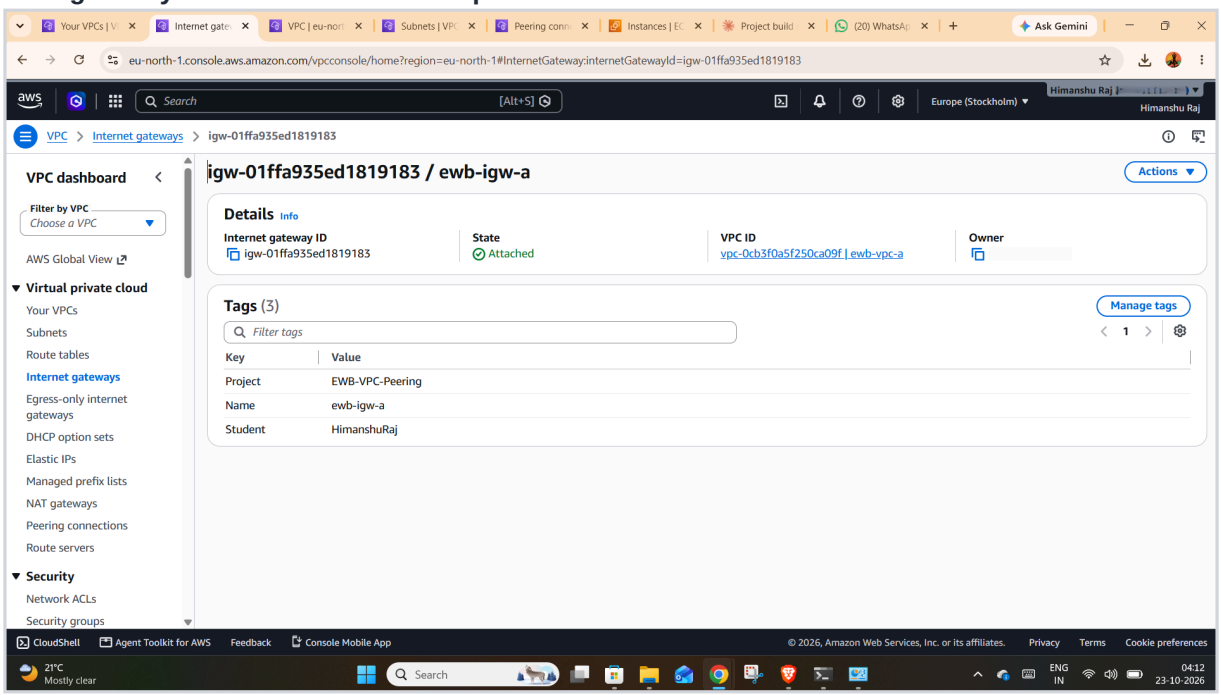
12. VPC peering deletion confirmation



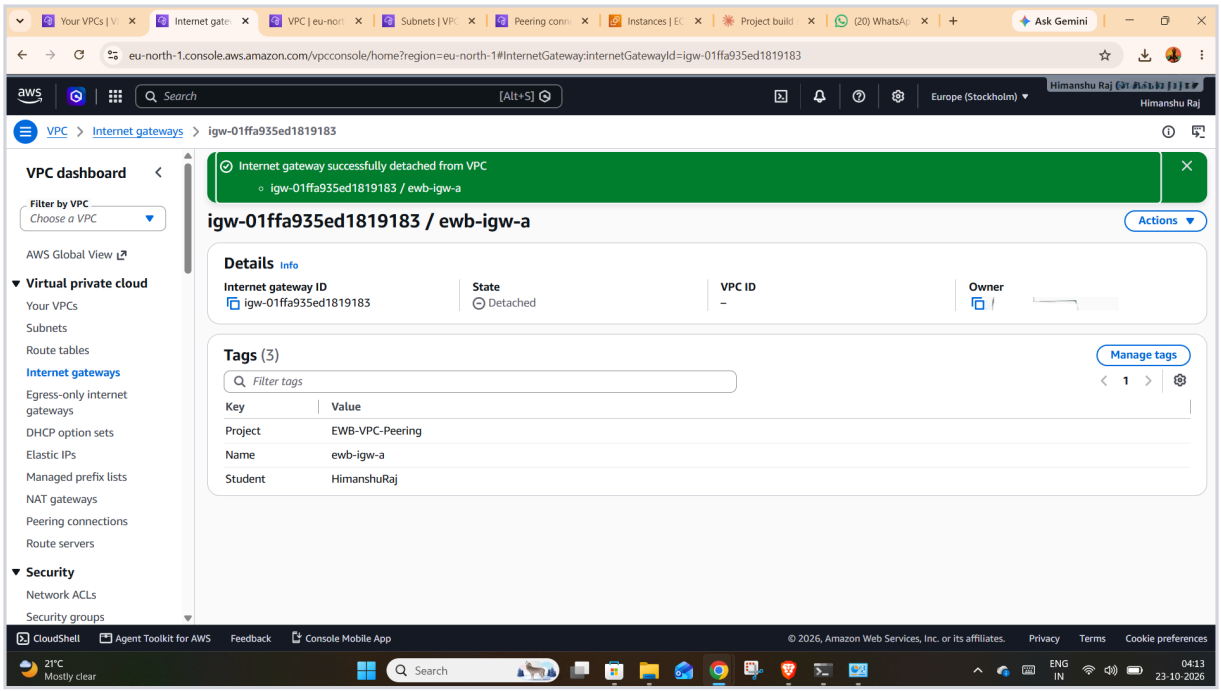
EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

13. Internet gateway attached before cleanup



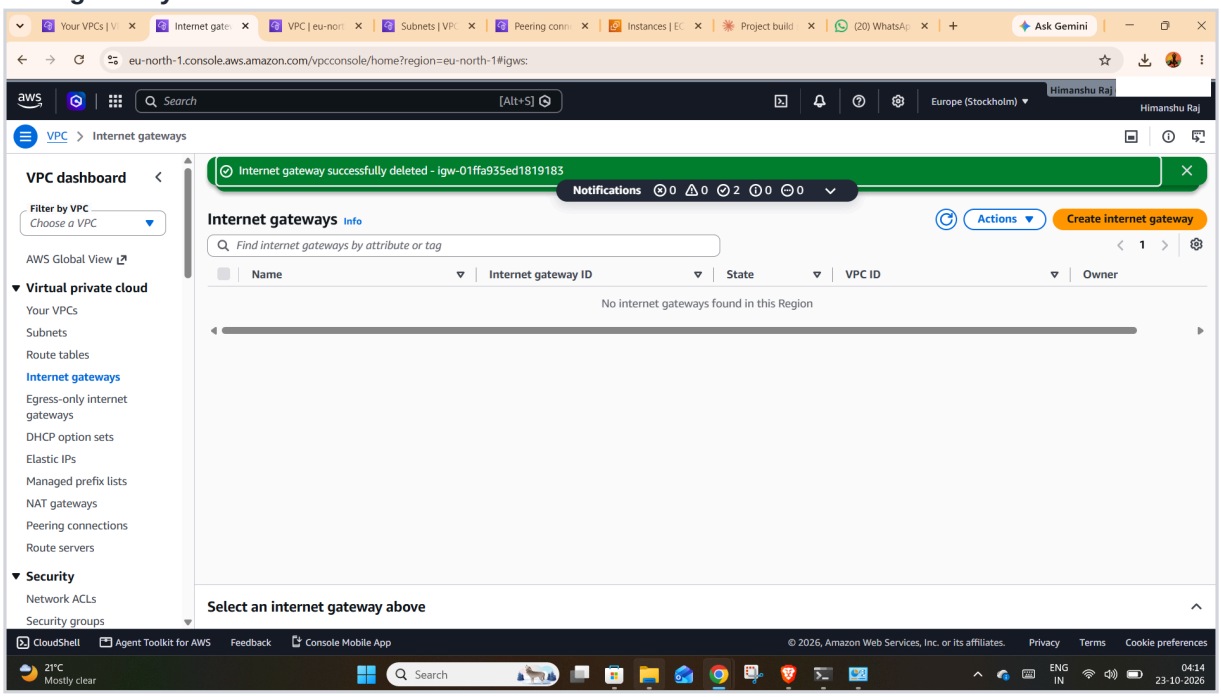
14. Internet gateway detached from VPC



EWB AWS VPC Peering | Cleanup Evidence

AWS Console screenshots supplied for this project. Account ID digits blurred; remaining evidence preserved.

15. Internet gateway deletion confirmation



11. Conclusion

This project successfully demonstrates the implementation and testing of VPC peering between two Amazon VPCs in the eu-north-1 (Europe – Stockholm) region. The environment was built according to the provided instructions, with a secure network design, correctly configured route tables, and security-group rules for controlled communication between the resources.

Connectivity tests were completed, including forward and reverse ping tests, private SSH access through the bastion host, and a controlled failure by removing the ICMP Echo Request rule followed by successful recovery. The results demonstrate private communication between the VPCs through peering while maintaining network security and isolation.

All project tasks have been completed, including resource creation, configuration, connectivity testing, controlled failure and recovery, evidence capture, and final cleanup. The cleanup evidence is included in this report, and the project is complete according to the provided instructions.

PROJECT COMPLETED SUCCESSFULLY

All required instructions and project objectives have been completed. The report includes the network design, configuration details, test results, troubleshooting evidence, and AWS resource-cleanup evidence.